Giving Claude Hands

Lucas J. Fawcett

Syracuse University

Abstract

This paper investigates methods for enabling Claude, a large language model developed by Anthropic, to interact with SolidWorks 2024 for autonomous 3D CAD part generation. Three integration methods were evaluated: VBA Macro, Python + COM, and a Local Model Context Protocol (MCP) Server. Each method was tasked with producing a parametric L-bracket containing extrusions, through-holes, fillets, chamfers, and material assignment. The VBA Macro method offered the simplest setup and most compact code (148 lines) but was the least geometrically complete, failing silently on key features due to late-binding limitations. The Python + COM method produced correct additive geometry but required eleven major revisions and could not execute any cut-extrude operation due to an undocumented parameter count mismatch in the SolidWorks API. The Local MCP Server produced the most complete part and was the only method to resolve the FeatureCut4 parameter mismatch through its iterative tool-call architecture, which allowed individual operations to be tested and retried based on API feedback. Following method selection, the MCP server underwent a systematic training phase using empirical diagnostic scripts that validated 22 of 26 tools by exhaustively testing method names, parameter counts, selection marks, and COM dispatch modes against live SolidWorks geometry. The trained system was then used to generate parts of increasing complexity, including a wireframe cube sculpture, a revolved and shelled geometric chalice, a 27-feature Cessna 172 model, and a NACA 2412 airfoil produced using both polygon and spline methods. Results demonstrate that an AI language model can function as an effective CAD operator when provided with appropriate tooling and an iterative feedback mechanism, though the absence of visual feedback remains the primary limitation, causing spatial errors such as disconnected features and incorrect cut depths to propagate undetected until user inspection.

Abstract 2
1. Introduction 5
1.1 Background 5
1.2 Problem Statement 5
1.3 Research Question 5
1.4 Objectives 6
1.4.1 6
1.4.2 6
2. Methodology 6
2.1 Experiment One (Testing Methods) 6
2.2 Method Setup 7
2.2.1 VBA Macro 7
2.2.2 Python + COM 9
2.2.3 Local MCP Server 10
2.3 Base Part Description 12
2.4 Continued Training 13
3. Results 14
3.1 Method Test Parts 14
3.1.1 VBA Macro 14
3.1.2 Python + COM 17
3.1.3 Local MCP Server 20
3.2 Training 23
3.2.1 Initial Tool Assessment 23
3.2.2 Diagnostic Script Methodology 23
3.2.3 Validating Existing Tools 24
3.2.4 Diagnosing and Fixing Failed Tools 25
3.2.5 Adding New Tools 27
3.2.6 Final Tool Status 28
3.3 Trained System Outputs 29
3.3.1 Bowl (Shell + Chamfer Test) 29
3.3.2 Frame Cube 30
3.3.3 Geometric Chalice 32
3.3.4 Cessna 172 33
3.3.5 NACA Airfoil - Polygon Method 36
3.3.6 NACA Airfoil - Spline Method 39
4. Conclusion 41
5. Future Work 43
6. References 45
Appendices 46
Appendix A 46
Appendix B 50
B.1 Imports and Unit Conversion 50
B.2 Design Parameters 50
B.3 COM Connection 50
B.4 Template Discovery 51
B.5 Equation Injection with Cascading Fallback 51
B.6 Boss-Extrude Operation 52
B.7 Cut-Extrude with InvokeTypes 52
B.8 Geometry Creation Example — Base Plate 53
B.9 Material Assignment 53
Appendix C 54
C.1 Claude Desktop Configuration 54
C.2 MCP Server Script (Solidworks_mcp_server.py) 54

C.2.1 Imports and COM Initialisation..... 54
C.2.2 COM Type Helpers..... 54
C.2.3 Async-to-COM Bridge..... 55
C.2.4 Sketch Selection..... 55
C.2.5 World-to-Sketch Coordinate Transform..... 55
C.2.6 Sketching on Model Faces..... 56
C.2.7 Boss-Extrude Operation..... 56
C.2.8 Cut-Extrude Operation..... 57
C.2.9 Edge-Dependent Features..... 58
C.2.10 Material Assignment..... 58
C.2.11 MCP Tool Registration..... 58
C.2.12 Server Entry Point..... 59

1. Introduction

1.1 Background

Artificial intelligence tools such as Claude are increasingly being explored in engineering workflows. In Computer-aided design (CAD), tools like SolidWorks [1] require precise actions, constraint management, and design intent, making them a strong test case for AI-assisted interaction.

SolidWorks is a leading 3D Computer-Aided Design (CAD) and engineering application used to create, simulate, publish, and manage data for product development. It operates on Windows, utilizing parametric, feature-based modeling to create 3D parts, assemblies, and 2D drawings commonly used in manufacturing and machinery design. [1]

Claude is a next-generation AI assistant and family of large language models (LLMs) developed by Anthropic, designed to be safe, accurate, and secure. It is widely used for creative writing, coding, complex reasoning, and analyzing large documents or images. Known for its "constitutional AI" training, Claude focuses on being helpful and harmless [2].

There are many methods that can be used to have Claude interact with SolidWorks. The three that are tested are VBA Macro, Python + COM, and a local MCP server. Additionally, the Claude Chrome extension has the ability to control mouse and keyboard inputs. This method was unable to be tested on SolidWorks as the Chrome extension can only view Chrome web tabs. However, this was tested using the web CAD tool Onshape.

Onshape is a cloud-native, browser-based 3D CAD and PDM software platform developed by PTC that allows teams to collaborate in real-time, functioning on any device without installation. It offers professional parametric modeling, built-in data management, and eliminates file-based, "check-in/check-out" bottlenecks. [3]

1.2 Problem Statement

Despite rapid advancements in AI, it is currently unclear how effectively Claude can translate ambiguous or high-level natural language prompts into accurate, fully constrained, and manufacturable 3D models within SolidWorks. Specifically, regarding maintaining the integrity of complex feature trees (sketches, extrusions, fillets, and holes) as well as accurately defining sketch relations, geometric tolerances, and parametric linkages without visual feedback of the 3D space.

1.3 Research Question

What method is the best for Claude to interact with SOLIDWORKS, and how effectively can Claude create parts in SolidWorks when given different types of prompts?

1.4 Objectives

1.4.1

To evaluate the technical feasibility of interaction methods, as well as the ease of setup, and determine which method is the best.

1.4.2

To determine how Claude performs using the best method when given a wide range of tasks and prompts.

2. Methodology

2.1 Experiment One (Testing Methods)

Claude was given the following prompt and told to ignore the drawing requirement. This was done to make the task simpler for Claude, as the engineering drawings are very visually based and end up being hard for an AI that can't see the desktop to create.

User Prompt

```
Create a fully parametric L-bracket model in SolidWorks and provide both the completed part file
(.SLDPRT) and a fully dimensioned drawing file (.SLDDRW).
The part should be a right-angle bracket consisting of a horizontal base plate and a vertical plate. The
base plate should be approximately 120 mm × 60 mm × 8 mm thick, and the vertical plate should extend 80
mm upward with the same thickness.
Include the following features:
Mounting holes on the base plate (Ø8–10 mm) created using Hole Wizard and positioned using proper sketch
relations and dimensions.
Two vertically aligned holes on the vertical plate (Ø6–8 mm), spaced evenly using symmetry or a linear
pattern.
A centered reinforcing rib between the base and vertical plate using the rib feature.
Fillets (3–8 mm radius) applied to internal corners.
Chamfers (1–2 mm) applied to select external edges.
Use linear patterns where appropriate instead of manually duplicating features.
The model must demonstrate proper SolidWorks practices:
All sketches must be fully defined.
Use parametric design with global variables (e.g., thickness, height, hole diameter) and equations to
control dimensions.
Features should be clearly named in the feature tree.
Assign a realistic material such as Aluminum 6061.
In addition to the part file, create a professional engineering drawing (.SLDDRW) that includes:
Standard orthographic views (front, top, right, and isometric).
Complete and clear dimensions for all critical features (overall size, hole locations, hole diameters,
rib thickness, etc.).
Proper use of centerlines, center marks, and hole callouts.
Tolerances where appropriate.
A title block with part name, material, and scale.
The final deliverables should include:
Fully defined SolidWorks part file (.SLDPRT)
Fully dimensioned drawing file (.SLDDRW)
The goal is to demonstrate strong foundational SolidWorks skills, proper design intent, and professional
drafting standards.
```

Claude was asked to complete this task using one of the specified methods (VBA Macro, Python + COM, and Local MCP). Claude was then helped in setting up these methods and given time to perform the task. The part files it created were then turned into engineering drawings by hand.

2.2 Method Setup

2.2.1 VBA Macro

The VBA Macro method was configured using SolidWorks 2024's built-in Visual Basic for Applications (VBA) editor, which ships as a standard component of all SolidWorks desktop installations and requires no external software, packages, or runtime environments. The VBA editor is accessed from

within SolidWorks via Tools → Macro → New, which creates a *.swp* macro file and opens the integrated development environment (IDE) directly inside the SolidWorks process.

A COM connection to SolidWorks was established using *GetObject(, "SldWorks.Application")*, which returns a VBA Object reference representing the top-level *ISldWorks* interface. Unlike the Python + COM method, which launches or connects to SolidWorks externally via Dispatch, the VBA macro runs in-process as a hosted component of SolidWorks itself. The active document was accessed via *swApp.ActiveDoc*, which returns a reference to the currently open part, and a type check *(swModel.GetType <> 1)* confirms the active document is a part file before proceeding. The user opens a blank part manually via File → New → Part → OK before running the macro.
Late binding was used exclusively, with all object variables declared as As Object rather than their strongly-typed equivalents (e.g., As *SldWorks.SldWorks*, As S*ldWorks.ModelDoc2*). This eliminated the need to configure type library references in the VBA editor's Tools → References dialog, which would otherwise require checking *SldWorks 20XX* Type Library and SolidWorks 20XX Constant Type Library. A consequence of late binding is that all SolidWorks API enumeration constants must be replaced with their numeric equivalents, as the VBA runtime cannot resolve symbolic enum names without the type library. The constants used were:

Symbolic Constant	Numeric Value	Usage
swEndCondBlind	0	Blind extrusion end condition
swEndCondThroughAll	1	Through-all end condition
swEndCondMidPlane	6	Mid-plane symmetric extrusion

Table 1: Constant their values, and uses as they relate to the VBA Macro method

All key dimensions were injected as named global variables into the SolidWorks Equations Manager using *GetEquationMgr()*, with equations added via the *Add2* method signature. SolidWorks uses metres internally, so all millimetre values were converted using

$$v(m) = mm \times 10^{-3}$$

before being passed to geometry-creation API methods such as *FeatureExtrusion3*, *CreateCornerRectangle*, and *CreateCircleByRadius*. The Equations Manager stores values in millimetres for user-facing readability, so equation strings were formatted as e.g., *"Thickness" = 8mm*.

The geometry-creation API method *FeatureExtrusion3* was used for all solid features , base plate, vertical plate, and reinforcing rib , accepting both blind extrusion (end condition 0) and mid-plane symmetric extrusion (end condition 6) via its 23-parameter signature. Holes were created using the inner-loop sketch technique, in which hole profiles (circles) are drawn inside the outer boundary profile (rectangle) within the same sketch. When *FeatureExtrusion3* processes such a sketch, the SolidWorks geometry kernel interprets inner closed loops as material removals, producing the extruded solid with holes in a single feature. Edge-dependent features (fillets via *FeatureFillet3* and chamfers via *InsertFeatureChamfer*) relied on *SelectByRay* for programmatic edge selection, which shoots a ray from a specified coordinate in a given direction and selects the nearest edge within a tolerance radius.

Material assignment was handled using *SetMaterialPropertyName2*, specifying the material database file (solidworks *materials.sldmat*) and material name (6061 Alloy). Multiple database name capitalisations were attempted to account for variation across SolidWorks installations and language packs.

Macros were saved as .swp files via the VBA editor's save function, with part output saved manually by the user as .SLDPRT after macro execution. The complete VBA macro implementing this method is provided in Appendix A.

2.2.2 Python + COM

The Python + COM method was configured using *Python 3.14* with the *pywin32* package [5] (installed via pip install pywin32) on the same Windows machine running SolidWorks 2024. The *pywin32* package provides the *win32com.client* module for COM dispatch and the pythoncom module for low-level COM type constants.

A COM connection to SolidWorks was established using *win32com.client.Dispatch("SldWorks.Application")*, which returns a Python proxy object representing the top-level *ISldWorks* interface. If the generic *ProgID* failed, version-specific alternatives (*SldWorks.Application.28* through .33, corresponding to SolidWorks 2020–2025) were attempted sequentially until a successful connection was made. The Visible property was set to True so the SolidWorks window remained visible during automation, and *UserControl* was set to True to prevent the application from terminating when the script disconnected. Late binding was used as the primary COM dispatch mode, resolving method names and parameter types at runtime via the *IDispatch* interface. A known limitation of late binding is its inability to distinguish between property access and method invocation for ambiguous COM member names, producing the error "Parameter not optional" (*HRESULT -2147352561*). This was addressed by calling *_FlagAsMethod()* on the exact dispatch wrapper object used for the subsequent method call. Because each access to a sub-object, such as *model.FeatureManager*

returns a new dispatch wrapper; the flag had to be applied immediately before every call on that same wrapper instance. When *_FlagAsMethod* was insufficient, *InvokeTypes()* was used as a lowest-level fallback, sending raw COM Invoke calls with explicit type constants (*VT_BOOL, VT_I4, VT_R8*) for every parameter.

Part template paths (*.PRTDOT*) were discovered through a three-tier strategy: querying SolidWorks user preferences via GetUserPreferenceStringValue, scanning installation directories across SolidWorks versions 2018–2026, and recursive filesystem search as a final fallback. The resolved path was *C:\ProgramData\SolidWorks\SOLIDWORKS 2023\templates\Part.PRTDOT.*
All key dimensions were injected as named global variables into the SolidWorks Equations Manager using *GetEquationMgr()*, with equation addition attempted via *Add3, Add2*, and *Add* method signatures in sequence to accommodate version differences. Custom Properties served as a fallback if the Equations Manager was inaccessible. SolidWorks uses metres internally, so all millimetre values were converted using

$$v(mm) = mm \times 10^{-3}$$

before being passed to API methods.

Scripts were stored at *C:\Users\lfawc\OneDrive\ClaudeSolidworks\PythonCOM* with part output directed to *C:\Users\lfawc\Documents\L_Bracket*. The complete Python script implementing this method is provided in Appendix B.

2.2.3 Local MCP Server

The MCP Server Bridge method was configured using Python 3.14 with the *pywin32* [5]and *mcp[server]* packages (installed via pip install *pywin32* "*mcp[server]*") on the same Windows machine running SolidWorks 2024. The *pywin32* package provides the *win32com.client* module for COM dispatch and the pythoncom module for low-level COM type constants. The *mcp[server]* package provides the *FastMCP* framework for implementing a Model Context Protocol (MCP) server [4] [6], which exposes Python functions as remotely callable tools over a structured JSON-RPC transport.

The architecture consists of three components connected in series: the Claude Desktop application, a locally hosted MCP server process, and the SolidWorks COM interface. Claude Desktop acts as the MCP client, launching the Python server script as a child process and communicating with it over standard input/output (*stdio*) using the MCP protocol. The server process holds a persistent COM connection to SolidWorks 2024 and translates incoming MCP tool calls into SolidWorks API method invocations. Unlike the VBA Macro method, which runs in-process as a hosted component of SolidWorks, and the Python + COM method, which runs as a standalone external script, the MCP Server Bridge runs as a continuously listening daemon process whose lifecycle is managed by Claude Desktop.

The user issues natural-language instructions to Claude, which the language model interprets and converts into structured tool calls with appropriate parameter values; the server executes these calls against the SolidWorks API and returns JSON-formatted results to Claude for further reasoning.

A COM connection to SolidWorks was established using *win32com.client.Dispatch*("*SldWorks.Application*"), identical to the Python + COM method. The Visible property was set to True so the SolidWorks window remained visible during automation. Late binding was used exclusively, with all method resolution performed at runtime via the *IDispatch* interface. SolidWorks must be running with an open part document before Claude Desktop launches the server process, as the COM dispatch call connects to an existing SolidWorks instance rather than launching a new one. The active document was accessed via *swApp.ActiveDoc*, which returns a proxy object representing the currently open part.

All SolidWorks API calls were executed on background threads via Python's *asyncio.run_in_executor()*, with *pythoncom.CoInitialize()* is called at the start of each executor thread to satisfy COM's apartment-threading requirements. Each MCP tool function is declared as an async coroutine decorated with *@mcp.tool()*, which registers it in the server's tool manifest. When Claude Desktop invokes a tool, the *FastMCP* framework deserialises the incoming JSON parameters, calls the corresponding async function, and serialises the return value back over *stdio*.

SolidWorks uses metres internally, so all millimetre values provided by Claude in natural-language instructions were converted using

$$v(m) = mm \times 10^{-3}$$

before being passed to geometry-creation API methods. This conversion was performed by Claude itself during parameter construction, as the tool descriptions specify that all dimensional inputs are in metres.

The geometry-creation API method *FeatureExtrusion2* was used for boss/base features, accepting a 20-parameter signature with blind extrusion (end condition 0) as the default. Cut features used *FeatureCut4*, which required exactly 27 parameters on SolidWorks 2024 SP5; this parameter count was determined empirically by executing a diagnostic script that tested all known *FeatureCut* method variants (*FeatureCut* through *FeatureCut4*) with varying parameter counts until a successful invocation was identified. Sketch selection prior to extrusion was handled by walking the feature tree via *FirstFeature/GetNextFeature* to locate the last *ProfileFeature* entry, then selecting it by name using *Extension.SelectByID2* with a null dispatch variant for the callout parameter. For cut operations, the sketch was re-selected immediately before each FeatureCut4 invocation to prevent selection loss between attempts.

Reference plane selection for sketching used *Extension.SelectByID2* with the plane name suffixed by " Plane" (e.g., "Front Plane") and entity type "PLANE". A null COM variant, constructed via

win32com.client.VARIANT(pythoncom.VT_DISPATCH, None). was required for the callout parameter to avoid type-mismatch errors on the *SelectByID2* interface.

Face-based sketching , required for placing holes on model faces rather than reference planes , used *Extension.SelectByRay* to programmatically select a face by shooting a ray from a specified coordinate in a given direction. Once a face was selected and a sketch opened via *SketchManager.InsertSketch(True)*, world-coordinate inputs were converted to the face's local sketch coordinate system using the sketch's *ModelToSketchTransform* matrix. The 4×4 transformation matrix was read via the transform's *ArrayData* property, and the world-to-sketch conversion was performed using manual matrix multiplication:

$$sx = a \cdot wx + d \cdot wy + g \cdot wz + tx,\ sy = b \cdot wx + e \cdot wy + h \cdot wz + ty,$$

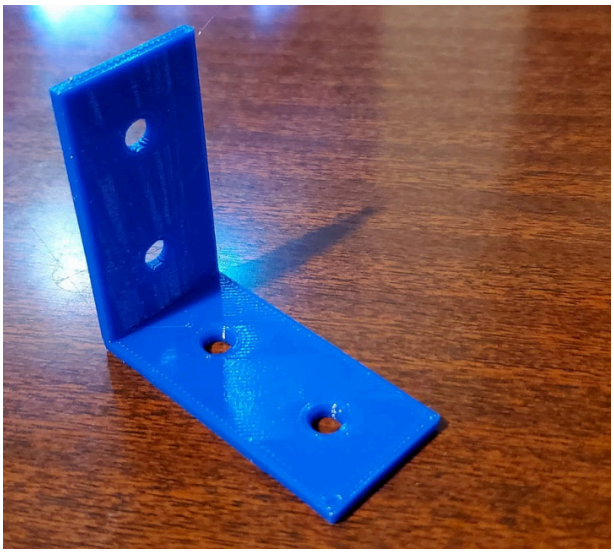
where "*a*" through "*i*" are rotation components and tx, ty are translation components of the transform. This approach was adopted over the *MathUtility.CreatePoint* COM method due to COM type-marshalling failures when constructing VARIANT arrays for the point coordinates.

Edge-dependent features (fillets via *FeatureFillet3* and chamfers via *InsertFeatureChamfer*) relied on *SelectByRay* for programmatic edge selection, with the append parameter set to True for multi-edge selection. Material assignment was handled using *SetMaterialPropertyName2*, specifying the material database file (solidworks *materials.sldmat*) and material name (e.g., "6061 Alloy").

The MCP server was registered with Claude Desktop by adding a JSON configuration block to the application's configuration file, located at the path opened by the Developer → Edit Config link within Claude Desktop's settings interface. The configuration specifies the full path to the Python interpreter executable and the server script as command-line arguments, along with the environment variable PYTHONUNBUFFERED=1 to ensure real-time stdio communication. The server script was stored at *C:\SolidWorksMCP\solidworks_mcp_server.py*, with part output saved via the SolidWorks *Extension.SaveAs3* method to user-specified paths. The server exposes 25 MCP tools spanning document management, sketch creation, feature operations, measurement queries, and file export. The complete Python server script implementing this method is provided in Appendix C.

2.3 Base Part Description

The part used in the base testing of the methods is a very common part that has become more widely 3D printed as of late. An L-bracket is a very good starting point for this research as it demonstrates Claude and AI's ability to create useful engineering parts.



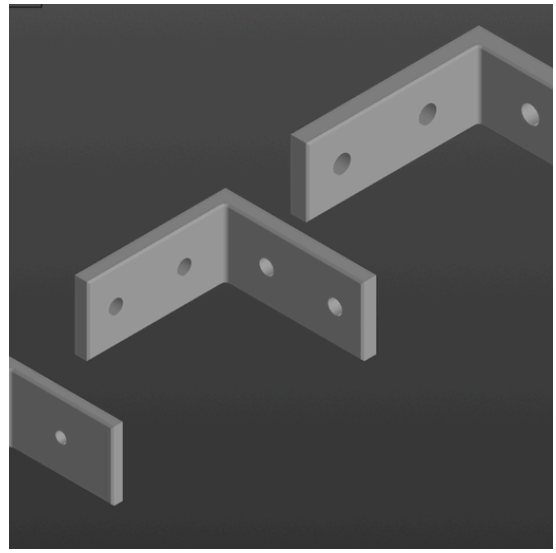


Figure 1: Example of 3D printable L-Brackets created by David McG. [8] (Left) and medyashkinvl [9] (Right).

2.4 Continued Training

Following the completion of Experiment One, the results from all three methods were evaluated to determine which approach offered the greatest potential for further development. The Local MCP Server was selected for continued training based on its ability to operate iteratively, receive feedback from the SolidWorks API between tool calls, and adapt its approach in real time, capabilities that the single-execution VBA Macro and Python + COM methods do not provide.

The training phase consisted of two objectives. The first was to validate and stabilise the existing tool set by systematically testing each of the 26 MCP tools against live SolidWorks geometry, identifying which tools functioned reliably, which failed due to incorrect COM parameters or dispatch issues, and fixing each failure through empirical diagnostic testing. The second objective was to extend the server with additional tools required for more complex part generation, including revolve, shell, linear pattern, circular pattern, and mirror operations, each of which required its own cycle of implementation, diagnostic testing, and parameter calibration.

The methodology used during training followed a repeatable four-stage process , test, diagnose, patch, verify , applied individually to each tool. When a tool failed, a purpose-built diagnostic script was executed against the live SolidWorks instance to exhaustively test all known method names, parameter counts, selection mark values, and COM dispatch modes until a working combination was identified. The confirmed parameters were then patched into the server and re-tested to verify correct operation. This process and its results are described in detail in Section 3.2.

Once the tool set was validated and extended, the trained MCP server was used to complete Experiment Two: the generation of more complex display parts demonstrating the full range of confirmed tools. The results of both experiments are presented in Section 3.

3. Results

3.1 Method Test Parts

Each method was able to produce a 3D printable part, although they all varied significantly in design and success. Each method was better at certain aspects of the CAD drafting; however, the Local MCP server proved to be the most successful. Note that all the parts were fully created by Claude using the directed method, but the drawings were created by me in order to show the parts more clearly and allow for better evaluation.

3.1.1 VBA Macro

VBA offers a streamlined and highly accessible way to automate SolidWorks because it is built directly into every desktop installation. There is no need to install additional software, packages, or runtime environments. Users can open the editor immediately through Tools → Macro → New and begin writing or running code without any prior setup. This simplicity stands in sharp contrast to the Python + COM approach, which requires installing Python, adding the pywin32 package, and configuring a working environment before any interaction with SolidWorks can begin.

Another key advantage is that VBA macros execute within the SolidWorks process itself rather than as an external application communicating through COM. Because of this in-process execution, there is no need to programmatically launch SolidWorks, manage whether the application is visible, or handle its lifecycle. The macro simply operates on the currently open document, avoiding the overhead of connection setup beyond a minimal object reference. This makes development more direct and reduces the potential points of failure that are common in external automation workflows.

Deployment is equally straightforward. A complete macro is contained in a single .swp file, which can be shared easily with other users. The recipient only needs to open SolidWorks, create or open a part, and run the macro through Tools → Macro → Run. There are no dependencies to install, no interpreter versions to align, and no environment variables or file paths to configure. This portability significantly lowers friction when distributing tools across teams.

The accessibility of VBA is further enhanced by the built-in macro recorder, which captures user actions and converts them into working code in real time. This allows even those without formal programming experience to generate functional scripts by performing tasks manually and then reviewing the recorded output. Combined with the fact that VBA is a widely known language with extensive documentation, it creates a low barrier to entry for engineers and designers who may not come from a software development background.

Working within the host application also results in more compact code. The full L-bracket macro required only 148 lines, compared to 749 lines for the Python + COM script and 458 lines for the MCP server. The reduction comes from eliminating the need for connection logic, application management, template discovery, and defensive COM dispatch handling such as _FlagAsMethod or InvokeTypes, all of which add significant overhead in external automation scripts.

Despite these advantages, the resulting part produced by the VBA macro method had several notable shortcomings. The most visible issue is the missing vertical plate. As noted on the engineering drawing, Claude was not able to generate the top plane of the bracket. The vertical plate sketch was drawn on the Front Plane and extruded using a mid-plane end condition to span the full 60 mm width, but the extrusion appears to have failed silently. Because the macro used On Error Resume Next style error suppression throughout and the FeatureExtrusion3 call returns Nothing rather than raising an error when it fails, the macro continued executing subsequent features without any indication that the vertical plate was not created. This is a fundamental weakness of the VBA approach , without visual feedback or robust error reporting, geometry failures can go undetected until the user inspects the final part.

The base plate, however, was created successfully with correct dimensions of 120 mm × 60 mm × 8 mm thick, and all four mounting holes are present at 15 mm from each edge as specified. The reinforcing rib also generated correctly as a triangular gusset with a 6 mm thickness and 40 mm leg dimensions, visible in both the front and isometric views. These features succeeded because they relied solely on FeatureExtrusion3 with simple blind or mid-plane end conditions and straightforward sketch geometry, confirming that the core extrusion workflow functions reliably in the late-bound VBA context.

The fillets and chamfers are absent from the final part. Both features depended on SelectByRay to programmatically select edges by shooting a ray from a specified coordinate toward the target geometry. Because the vertical plate was not generated, the internal corner edge where fillets were intended to be applied did not exist. Even for edges that did exist on the base plate and rib, the ray origin coordinates and directions had to be calculated manually based on assumed geometry positions. If any dimension is slightly off or the model topology differs from what the macro expects, the ray misses the target edge entirely and the feature is silently skipped. This contrasts with the MCP server method, where Claude could iteratively attempt edge selection and adjust coordinates based on feedback from the API, and with the Python + COM method, which had access to more robust error diagnostics.

The most significant technical limitation was the complete failure of all FeatureCut method variants when called through late binding. As described in Section 2.2.1, late binding was used exclusively to avoid type library configuration, but a consequence of this is that method parameter mismatches are not detected at compile time. Multiple variants, including *FeatureCut4*, *FeatureCut3* with different parameter counts, and the original *FeatureCut* all produced Runtime Error 449, indicating

parameter count mismatches. Unlike the Python + COM method, which resolved similar issues using *_FlagAsMethod()* and *InvokeTypes()* with explicit COM type constants, VBA provides no equivalent low-level mechanism to override how COM dispatch calls are constructed. This forced the use of the inner-loop sketch technique, where hole circles are drawn inside the outer rectangle within the same sketch so that a single extrusion produces solids with holes already defined. While effective for the base plate holes, this technique limits the ability to create cuts on non-planar faces, cuts at arbitrary angles, or features added to existing geometry after initial creation. It is worth noting that this is likely an environment-specific limitation rather than a fundamental flaw of VBA , enabling the SolidWorks type library references for early binding would have provided compile-time parameter validation and may have resolved the FeatureCut errors entirely, but these references were not available in the test environment.

A secondary limitation was the instability of COM object references during macro execution. Object variables such as *swApp*, *swModel*, and *swFeatMgr* became stale after repeated sketch open and close cycles, producing Runtime Error 91. This required re-acquiring all object references via GetObject and ActiveDoc after every sketch operation, adding repetitive boilerplate code throughout the macro. While this pattern reliably prevented crashes, it is not well-documented in the SolidWorks API reference and represents a hidden complexity that undermines the apparent simplicity of VBA macro development.

In summary, the VBA macro method excelled in accessibility, setup simplicity, and code compactness, but struggled with API reliability. Of the eight required features , base plate, vertical plate, base holes, vertical plate holes, reinforcing rib, fillets, chamfers, and material assignment , the macro successfully produced three solid features (base plate with holes, rib) while the vertical plate with its holes failed silently, and the edge-dependent features (fillets and chamfers) could not be applied. The method's inability to use FeatureCut and its reliance on coordinate-based edge selection without feedback made it the least geometrically complete of the three methods tested.

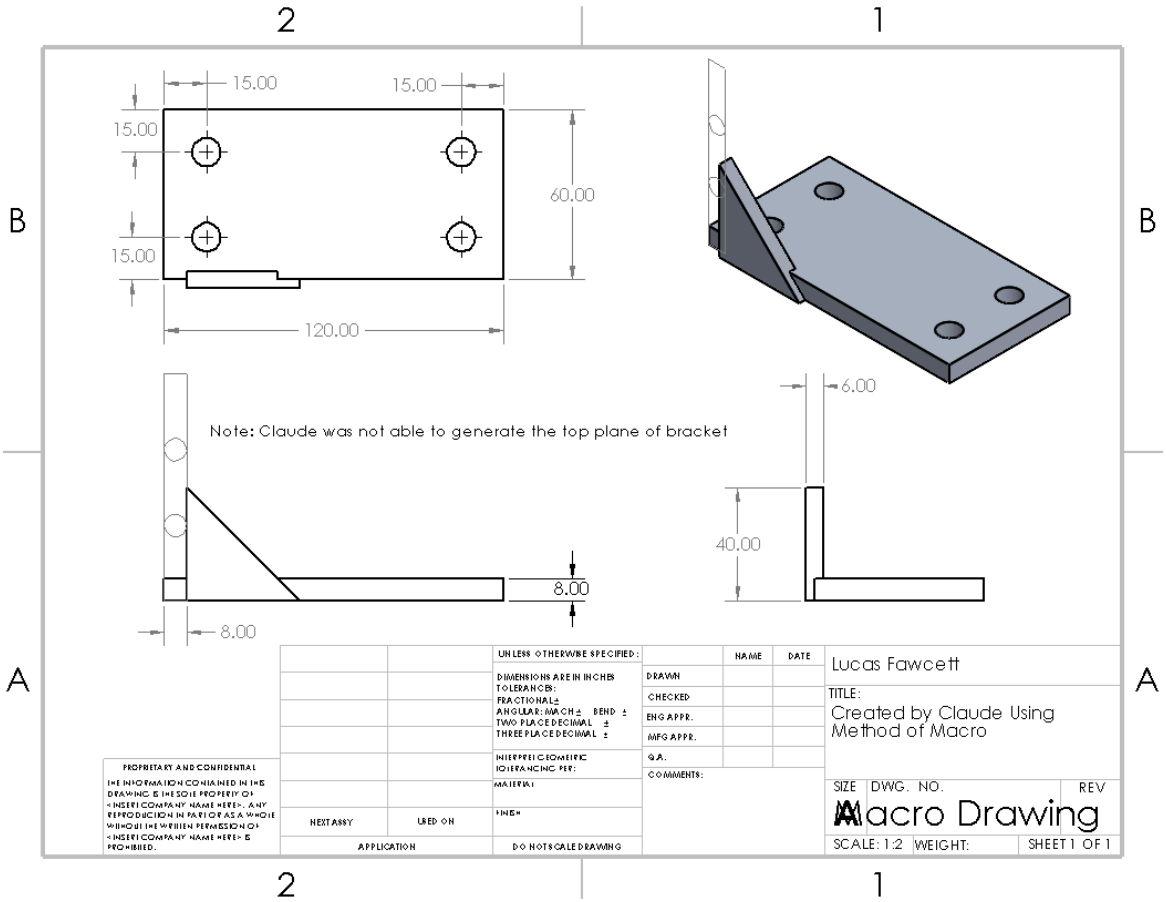


Figure 2: VBA Macro created L-Bracket. The holes on the vertical part were drawn too large and, as a result, sliced the entire vertical plate off. A triangle was added as support, which was not asked for and was generated on its own.

3.1.2 Python + COM

The Python + COM method produced a part file containing the correct overall geometry, base plate, vertical plate, reinforcing rib, fillets, and chamfers, but failed to create any of the six through-hole features. The script went through eleven major revisions across an extended debugging session with Claude before reaching a final state, making it by far the most labour-intensive method to develop.

Additive feature creation was fully successful. The *FeatureExtrusion2* method, called through late binding with *_FlagAsMethod* applied to the *FeatureManager* wrapper as described in Section 2.2.2, reliably produced all three boss-extrude features: the 120 × 60 × 8 mm base plate on the Top Plane, the 120 × 80 × 8 mm vertical plate on the Front Plane, and the 6 mm mid-plane reinforcing rib on the Right Plane. Sketch geometry creation via *SketchManager* methods (*CreateCenterRectangle*, *CreateLine*, *CreateCircle*) worked without issue throughout all revisions. The Equations Manager accepted all eleven global variables via the *Add3*/*Add2* cascading fallback described in Appendix B.5, fillets were applied via

FeatureFillet3, chamfers via *InsertFeatureChamfer*, and Aluminum 6061 Alloy was assigned via *SetMaterialPropertyName2*.

The critical failure was the complete inability to execute any cut-extrude operation. All six hole sketches were successfully created , they appeared in the feature tree as Sketch4 through Sketch9 with type *ProfileFeature* , but every attempt to convert them into cut features produced the COM error *DISP_E_PARAMNOTOPTIONAL* (HRESULT -2147352561, "Parameter not optional"). This error indicates that the COM dispatch layer rejected the method call before SolidWorks processed it, meaning the parameter signature being sent did not match what the *IFeatureManager* interface expected. Claude attempted five distinct calling strategies across the eleven revisions:

- ***InvokeTypes* with *FeatureCut4* (24 parameters)** — the lowest-level COM call available in *pywin32*, specifying explicit type constants (*VT_BOOL*, *VT_I4*, *VT_R8*) for every parameter as shown in Appendix B.7. Failed with the same *DISP_E_PARAMNOTOPTIONAL* error.
- ***InvokeTypes* with *FeatureCut3* (20 parameters)** — an older method variant with fewer parameters. Same error.
- **Late-binding calls to *FeatureCut4* and *FeatureCut3*** — using *_FlagAsMethod* on the *FeatureManager* wrapper. Same error.
- **Explicit VARIANT-wrapped parameters** — each argument individually wrapped using *win32com.client.VARIANT(pythoncom.VT_BOOL, True)* to force correct type marshalling. Same error.
- **VBA macro via *RunMacro2*** — a complete *.swb* macro file was written to disk containing native VBA code that performed the sketch-circle-close-cut sequence entirely within VBA. The macro was invoked from Python using *sw.RunMacro2()*. This also failed, returning False with error code 0, suggesting the macro itself encountered the same *FeatureCut* parameter mismatch at runtime.

The fact that *FeatureExtrusion2* succeeded while *FeatureCut4* failed under identical calling conditions points to a difference in how the two methods are registered in the SolidWorks COM type library. The MCP server method, described in Section 2.2.3, later determined empirically that *FeatureCut4* on SolidWorks 2024 SP5 requires exactly 27 parameters, three more than the 24-parameter signature documented in the SolidWorks API reference and used by the Python + COM script. This parameter count discrepancy, invisible to late binding and undetectable without type library inspection, is the most likely root cause of the persistent failure.

A secondary issue compounded the debugging effort: *pywin32*'s late-binding dispatch could not correctly invoke zero-argument COM methods that return values, such as *Extension.GetLastFeatureAdded()*, *model.FirstFeature()*, *feat.GetNextFeature()*, and

feat.GetTypeName2(). Python's COM runtime treated these as property accesses rather than method calls, producing the error *DISP_E_MEMBERNOTFOUND* (-2147352573, "Member not found"). The *_FlagAsMethod* function resolved this for methods called directly on the model object, but did not persist on sub-objects such as *model.Extension*, because each property access to *.Extension* returned a new COM dispatch wrapper on which no methods had been flagged. Claude identified this in revision 10 and introduced a caching pattern, storing *EXT = model.Extension* once and reusing the cached reference, but *GetLastFeatureAdded* continued to fail even on the cached object.

The practical consequence was twofold. First, feature renaming silently failed throughout all revisions: the *rename_last* function caught the exception and continued, leaving every feature with its default SolidWorks name (*Boss-Extrude1*, *Boss-Extrude2*, etc.) rather than the intended descriptive names (Base Plate, Vertical Plate, Reinforcing Rib). This was not discovered until revision 9 added diagnostic output, because the *try/except* block masked the error. Second, the script could not locate hole sketches after closing them, since the primary lookup method (*GetLastFeatureAdded*) was broken. A tree-walk fallback using *FirstFeature*/*GetNextFeature* was added in revision 10, which did work once those methods were properly flagged, but by that point the sketch could be found and selected, the cut operation itself still failed.

A third issue affected file output. The *SaveAs* fallback method in early revisions passed *swDocPART = 1* as the version parameter to *SaveAs4* instead of *swSaveAsCurrentVersion = 0*, causing the file to not be written to disk despite the method not raising an exception. This was identified in revision 11 but may also have been affected by OneDrive folder redirection, which caused the expected output path (*~/Documents/L_Bracket/*) to differ from the actual filesystem location. The drawing script consequently failed to locate the part file and exited before producing any output.

Unlike the VBA method, which worked around *FeatureCut* failures by drawing hole profiles as inner loops within extrusion sketches, the Python + COM script did not adopt this workaround. Claude instead pursued the direct *FeatureCut* approach across all eleven revisions, investing significant effort into COM dispatch diagnostics, type library inspection, *InvokeTypes* parameter specification, VARIANT wrapping, and VBA macro delegation, none of which resolved the underlying parameter count mismatch. The inner-loop technique used by the VBA method would have been equally applicable here, as *SketchManager.CreateCircle* was confirmed working, but this alternative was never attempted.

The final delivered part file contains the base plate, vertical plate, reinforcing rib, fillets, chamfers, material assignment, and eleven global variables, but lacks all six through-holes and has default feature names throughout. The six hole sketches remain in the feature tree and can be manually converted to cut-extrude features within SolidWorks. No engineering drawing was produced. The complete script in its final revision state is provided in Appendix B.

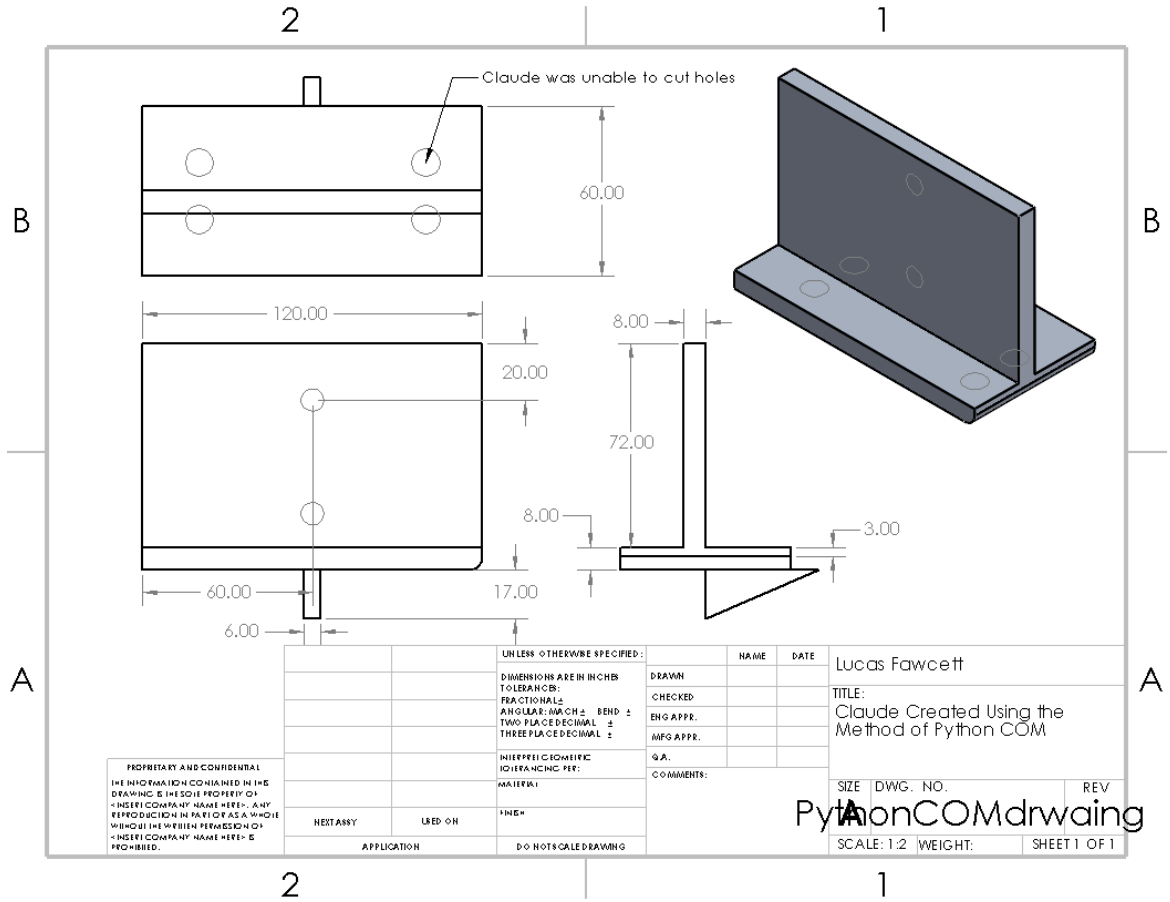


Figure 3: Python + COM created L-Bracket. The bracket is a non centered T-bracket, with a triangle placed randomly on the underside of the base. Holes were drawn but a cut was not performed.

3.1.3 Local MCP Server

The MCP server method produced the most geometrically complete L-bracket of the three methods, successfully generating the base plate, vertical plate, all six through-holes, a fillet on the internal corner, and material assignment.

The L-bracket body was created in a single extrusion using the *sketch_polygon* tool, which drew the complete L-shaped profile as a six-vertex closed polyline on the Front Plane and extruded it 60 mm via *FeatureExtrusion2* with 20 parameters. This produced the base plate (120 × 60 × 8 mm) and vertical plate (8 × 60 × 80 mm) as a single monolithic feature, avoiding the two-extrusion approach used by both alternative methods.

All six through-holes were successfully created using the *sketch_circle_on_face* tool, which selects a model face by ray (*Extension.SelectByRay*) and draws a circle at a specified world coordinate.

The world-to-sketch coordinate conversion was handled by reading the active sketch's *ModelToSketchTransform* matrix via its *ArrayData* property and performing manual matrix multiplication:

$$sx = a \cdot wx + d \cdot wy + g \cdot wz + tx,\ sy = b \cdot wx + e \cdot wy + h \cdot wz + ty.$$

This approach was adopted after the initial implementation using *MathUtility.CreatePoint* failed due to COM type-marshalling errors when constructing *VARIANT* arrays for the point coordinates. The four base plate holes (Ø10 mm) were cut using *FeatureCut4* with 27 parameters, and the two vertical plate holes (Ø6 mm) were cut using the same method. All six cuts produced *ICE*-type features in the feature tree and resulted in visible through-holes in the final geometry.

The 27-parameter signature for *FeatureCut4* on SolidWorks 2024 SP5 was determined empirically by running a diagnostic script (*test_cut.py*) that tested all known *FeatureCut* method variants, *FeatureCut* (13 parameters), *FeatureCut2* (18 parameters), *FeatureCut3* (20 parameters), and *FeatureCut4* with parameter counts ranging from 24 to 28. Every variant except *FeatureCut4* with exactly 27 parameters returned HRESULT -2147352561 (*DISP_E_PARAMNOTOPTIONAL*). The sketch had to be re-selected by name via *Extension.SelectByID2* immediately before each *FeatureCut4* call, because failed COM method invocations clear the active selection. Both flip directions (*reverse = True* and *reverse = False*) were attempted automatically for each cut, since the cut direction relative to the sketch normal varies depending on which face the sketch was drawn on.

A fillet was successfully applied to the internal corner edge where the base plate meets the vertical plate, using *FeatureFillet3* with 14 parameters after selecting the edge via *SelectByRay*. A 3 mm radius was used after an initial 5 mm radius attempt failed, likely because the radius exceeded the minimum adjacent face dimension on the 8 mm thick walls. The chamfer operation caused the MCP server to time out and crash before completing. Material assignment (*SetMaterialPropertyName2* with "6061 Alloy") had been confirmed working in earlier testing but could not be applied after the server crash. The user was advised to complete these two features manually.

The primary technical challenge was face-based sketch positioning. Early attempts placed hole circles at incorrect positions because the face's local sketch coordinate system differs from the global model coordinate system, and the mapping varies depending on face orientation. Circles intended for the top face of the base plate appeared floating in empty space, and cuts applied to incorrectly positioned circles either failed or removed entire walls of the part. This was resolved by implementing the *ModelToSketchTransform* matrix approach described above, which works correctly for any face in any orientation.

A secondary challenge was the COM connection lifecycle. The MCP server process connects to SolidWorks via *win32com.client.Dispatch("SldWorks.Application")* when it first receives a tool call. If

Claude Desktop is restarted, the server process is terminated and relaunched, but the new process may fail to connect to the running SolidWorks instance, producing *SldWorks.Application.ActiveDoc* errors on every subsequent call. The reliable workaround was to close SolidWorks entirely, quit Claude Desktop, reopen SolidWorks with a blank part, and only then reopen Claude Desktop, ensuring the server process connects to a fresh COM instance during its initialisation.

A third challenge was the *_FlagAsMethod* requirement for certain COM methods. Late-binding dispatch in *pywin32* cannot distinguish between property access and method invocation for ambiguous COM member names. The *InsertFeatureChamfer* method on the *FeatureManager* object, for example, was interpreted as a property access rather than a method call, producing "Unknown name" errors. Adding *fm._FlagAsMethod("InsertFeatureChamfer")* immediately before the call resolved this.

Of the eight required features, base plate, vertical plate, base holes, vertical plate holes, reinforcing rib, fillets, chamfers, and material assignment, the server successfully produced six (base plate, vertical plate, all holes, and fillet), partially completed one (chamfer was confirmed working in isolation but caused a server crash during the L-bracket build), and did not attempt the reinforcing rib.

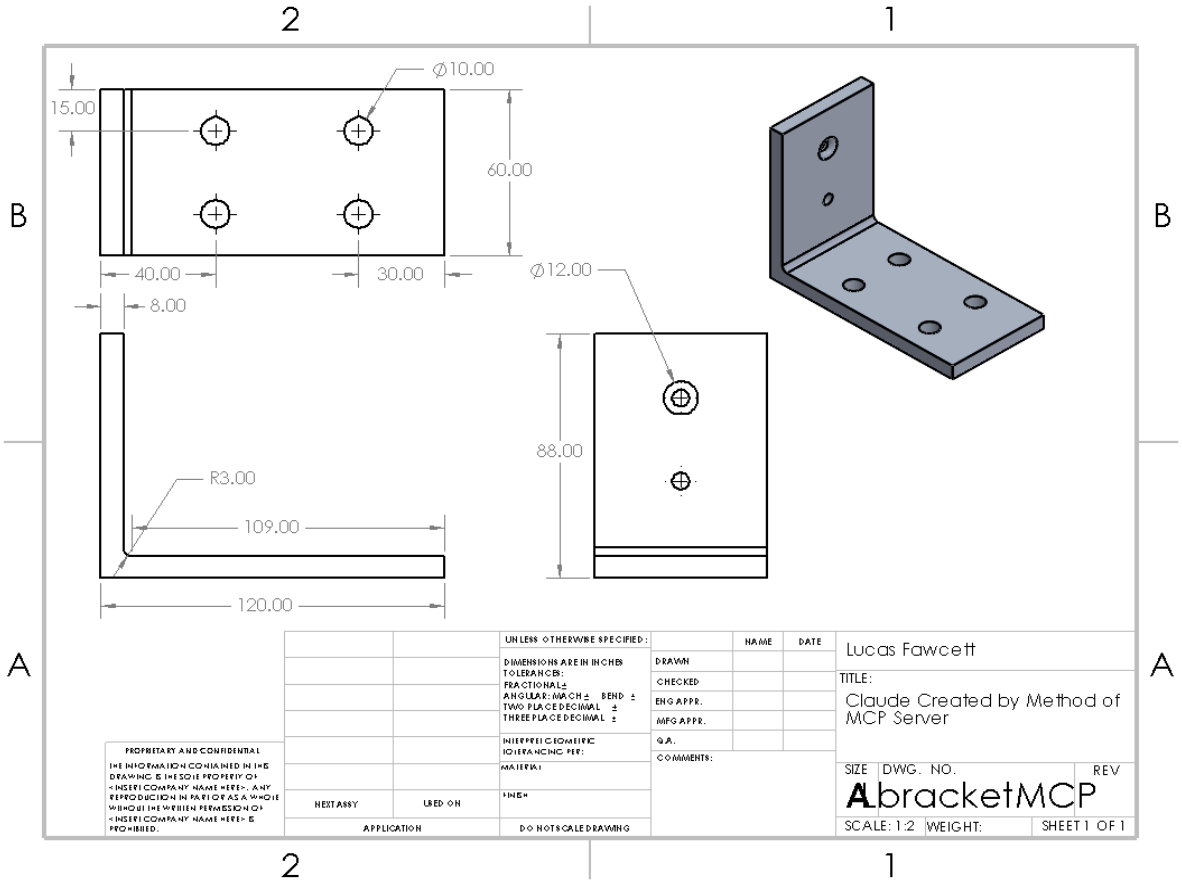


Figure 4: MCP Server created part. Only notable issue was a failure to perform a fillet or chamfer on the edges

3.2 Training

Of the three methods tested, the Local MCP Server produced the most complete geometry and was the only method to successfully resolve the *FeatureCut4* parameter mismatch that prevented hole creation in both alternative methods. Its iterative tool-call workflow, which allows individual operations to be tested, inspected, and retried, made it the most suitable candidate for continued development. The following subsections describe the process used to validate and extend the MCP server's tool set.

3.2.1 Initial Tool Assessment

The MCP server was deployed with 25 tools spanning document management, sketch creation, feature operations, measurement queries, and file export. After the L-bracket build, Claude was asked to provide an honest assessment of each tool's reliability:

User Prompt

```
"Can you give me a list of all the tools you have that are used in SolidWorks and rank them on how
confident you are in making them work?"
```

Claude categorised the tools into four tiers based on observed behaviour during the L-bracket build. Tools rated as confirmed working included all sketching operations on reference planes, *extrude_boss*, *set_material*, *save_document*, and *list_features*. Tools rated high confidence but requiring care included *extrude_cut*, *sketch_circle_on_face*, and *select_by_ray*. Partially working tools included *fillet_edges* and *extrude_cut_through_all*. Tools rated as untested or failed included *chamfer_edges*, *get_mass_properties*, *get_bounding_box*, and *new_part*. This assessment identified which tools could be relied upon immediately and which required further investigation before they could be used in production builds. The user then directed the training priorities:

User Prompt

```
"Let's focus on these tools: extrude_cut_through_all, get_mass_properties, fillet on multiple edges in
one batch. Should-add: linear pattern, circular pattern, mirror, revolve."
```

3.2.2 Diagnostic Script Methodology

The central technique developed during training was an empirical parameter discovery process using purpose-built diagnostic scripts. When a tool failed, a Python script was written that systematically tested every known combination of method name, parameter count, COM

object, selection mark value, and boolean flag against the live SolidWorks instance, reporting which combination produced a successful result.

Each diagnostic script followed the same structure. First, a test part was created or opened containing the prerequisite geometry (e.g., a box with a hole for testing patterns, a solid block for testing shell). Second, the script iterated through candidate method names on both the *FeatureManager* and *ModelDoc2* objects, for example, *InsertFeatureChamfer*, *InsertFeatureChamfer2*, and *InsertFeatureChamfer3*, applying *_FlagAsMethod* before each call to ensure COM dispatch treated the member as a method invocation rather than a property access. Third, for each method name, the script tested multiple parameter counts by appending or removing trailing boolean or integer values, since the SolidWorks API documentation does not always reflect the exact parameter count required by a specific version and service pack [7]. Fourth, for selection-dependent features such as mirror and pattern operations, the script tested all permutations of selection mark values (0, 1, 2, 4, 8, 16, 32, 64) for both the feature and the reference entity, since incorrect mark values cause the API call to silently return *None* without raising an error. Finally, when late-binding dispatch failed entirely, the script attempted raw COM calls via *oleobj.InvokeTypes()* with explicit type constants (*VT_BOOL*, *VT_I4*, *VT_R8*, *VT_DISPATCH*) for every parameter, bypassing all *pywin32* abstraction. A representative example is the *FeatureCut4* diagnostic output, which tested the method with parameter counts from 13 to 28:

Diagnostic Output

```
FeatureCut (13p, flip=F) → (-2147352561, 'Parameter not optional.') FeatureCut (13p, flip=T) →
(-2147352561, 'Parameter not optional.') ... FeatureCut4 (26p) → (-2147352561, 'Parameter not
optional.') FeatureCut4 (26p, flip) → (-2147352561, 'Parameter not optional.') SUCCESS: FeatureCut4
(27p)
```

The output of each diagnostic was a single confirmed working combination, method name, parameter count, parameter values, and selection marks, which was then patched into the MCP server's helper function. This approach treated the SolidWorks COM API as a black box and discovered correct calling conventions through exhaustive testing rather than documentation.

3.2.3 Validating Existing Tools

Three tools that had shown inconsistent behaviour during the L-bracket build were tested on a clean part (a 100 × 50 × 60 mm box with a single through-hole):

- *extrude_cut_through_all* used the same *FeatureCut4* with 27 parameters as the blind cut but with end condition 1 (*swEndCondThroughAll*) instead of 0 (*swEndCondBlind*) [7]. It passed on the first attempt.
- *get_mass_properties* had returned errors during the L-bracket build. After assigning material via *SetMaterialPropertyName2* and calling *Extension.GetMassProperties2*, it returned correct values for mass (0.799 kg), volume, surface area, and centre of mass.
- *fillet_edges* had succeeded once during the L-bracket build with a single edge. Testing confirmed it also worked on multiple edges selected via *select_by_ray* with the *append* flag set to *True*.

3.2.4 Diagnosing and Fixing Failed Tools

Four tools required diagnostic investigation. For each tool, the user was asked to run a diagnostic script and paste the output, which Claude then used to identify the working parameters and generate a patch script.

Chamfer — The *InsertFeatureChamfer* method on the *FeatureManager* object was not recognised by COM late-binding dispatch, producing "Unknown name" errors. The diagnostic output revealed the fix:

Diagnostic Output

```
fm.InsertFeatureChamfer(8p) → None SUCCESS: fm.InsertFeatureChamfer(8p) → <COMObject>
```

Adding fm._FlagAsMethod("InsertFeatureChamfer") before the call resolved the dispatch ambiguity. The first invocation returned None but still applied the chamfer (a pattern seen with several SolidWorks API methods), and subsequent calls returned a COM object. The method accepted 8 parameters: (4, 1, distance, 0, 0, 0, 0, 0).

Shell — No *InsertFeatureShell* method existed on the *FeatureManager* object [7]. The diagnostic tested the method on the *ModelDoc2* object instead and found that *doc.InsertFeatureShell(thickness, False)* executed successfully. However, the method returned *None* rather than a feature object, which caused the server's error-checking logic to incorrectly report failure. The user confirmed the shell had actually been applied:

User Prompt

```
"It did shell it though if that changes anything"
```

The fix was twofold: move the call from fm to doc, add doc._FlagAsMethod("InsertFeatureShell"), and remove the None-check that was treating a successful operation as a failure.

Mirror — Initial testing with fm.InsertMirrorFeature2 with 5 parameters returned None without creating geometry. Claude asked the user to record a macro of the manual mirror operation:

Claude Responds

```
"Go to Tools → Macro → Record, do the mirror manually, then Tools → Macro → Stop. Paste everything
you see in the code window."
```

The recorded VBA code revealed that SolidWorks used FeatureManager.InsertMirrorFeature (not InsertMirrorFeature2) [7]. A focused diagnostic then tested fm.InsertMirrorFeature with 4 boolean parameters across all permutations of feature selection mark and plane selection mark, using feature tree count verification to confirm actual geometry creation:

Diagnostic Output

```
SUCCESS: fm.InsertMirrorFeature([False, False, False, False]) marks f=1 p=2 (features 21→22)
```

Every previous attempt had used incorrect parameter values (True instead of False), incorrect mark values (4 instead of 1 for the feature, 1 instead of 2 for the plane), or the incorrect method variant (InsertMirrorFeature2 instead of InsertMirrorFeature).

Revolve — The FeatureRevolve2 method with 20 parameters created surface bodies (RevolvRefSurf) rather than solid bodies [7], regardless of which boolean parameters were toggled. Multiple rounds of diagnostics were required. The first tested FeatureRevolve2 with parameters 14–20 toggled in all combinations, all produced surface revolves. The second tested toggling parameters 3–6 and 11–13, still surface revolves. The breakthrough came when the older FeatureRevolve method was tested with parameter counts from 4 to 12:

Diagnostic Output

```
FeatureRevolve(5p) → 'Parameter not optional.' FeatureRevolve(6p) → 'Parameter not optional.'
FeatureRevolve(7p) → 'Parameter not optional.' FeatureRevolve(8p) → bodies=1, type=RevolvRefSurf
SUCCESS!
```

fm.FeatureRevolve(True, reverse, False, False, False, False, 0, 0) with _FlagAsMethod created a solid body. This method performs a default 360-degree rotation with no angle parameter. Creating the revolve sketch also required a new tool (sketch_revolve_profile) that draws both a

centreline axis via CreateCenterLine and a closed profile polygon via CreateLine within a single sketch, since the existing sketch tools auto-closed after each individual shape.

3.2.5 Adding New Tools

Four tools were identified as essential for complex part generation and added to the server via patch scripts that inserted helper functions and MCP tool registrations into the existing server file:

Linear pattern — *fm.FeatureLinearPattern3* with 10 parameters, requiring the feature to be patterned selected at mark 4 and a direction-defining edge selected at mark 1 via *SelectByRay* [7]. Confirmed by diagnostic and verified with a 3-instance hole pattern.

Circular pattern — *fm.FeatureCircularPattern4* with 7 parameters was identified by diagnostic as accepting the call without error [7], but the tool did not produce visible geometry in the feature tree during live testing. This remained the only unresolved tool at the end of the training phase.

Mirror — *fm.InsertMirrorFeature* with 4 parameters, as described in Section 3.2.4. Verified by creating a centred box with an off-centre hole and mirroring the cut feature about the Right Plane, producing a *MirrorPattern* feature in the tree.

Revolve — *fm.FeatureRevolve* with 8 parameters, as described in Section 3.2.4. Verified by creating a rectangular profile with a centreline axis and revolving it to produce a solid ring with correct mass properties.

3.2.6 Final Tool Status

At the conclusion of the training phase, 22 of the 26 tools (including the 4 newly added tools) were confirmed working through empirical testing. The confirmed tools and their validated API methods are summarised in Table 2.

Tool	API Method	Parameters	Status
sketch_rectangle	SketchManager.CreateCornerRectangle	—	Confirmed
sketch_circle	SketchManager.CreateCircleByRadius	—	Confirmed
sketch_line	SketchManager.CreateLine	—	Confirmed
sketch_polygon	SketchManager.CreateLine (loop)	—	Confirmed
sketch_circle_on_face	SelectByRay + ModelToSketchTransform	—	Confirmed
sketch_rect_on_face	SelectByRay + ModelToSketchTransform	—	Confirmed
sketch_revolve_profile	CreateCenterLine + CreateLine (loop)	—	Confirmed
extrude_boss	fm.FeatureExtrusion2	20	Confirmed
extrude_cut	fm.FeatureCut4	27	Confirmed
extrude_cut_through_all	fm.FeatureCut4 (end condition 1)	27	Confirmed
revolve	fm.FeatureRevolve	8	Confirmed
shell	doc.InsertFeatureShell	2	Confirmed
fillet_edges	fm.FeatureFillet3	14	Confirmed
chamfer_edges	fm.InsertFeatureChamfer	8	Confirmed
linear_pattern	fm.FeatureLinearPattern3	10	Confirmed
mirror	fm.InsertMirrorFeature	4	Confirmed
select_by_ray	Extension.SelectByRay	—	Confirmed
set_material	SetMaterialPropertyName2	—	Confirmed
get_mass_properties	Extension.GetMassProperties2	—	Confirmed
save_document	Save / Extension.SaveAs3	—	Confirmed
export_step / export_stl	Extension.SaveAs3	—	Confirmed
list_features	FirstFeature / GetNextFeature	—	Confirmed
delete_feature	SelectByID2 + DeleteSelection2	—	Confirmed
circular_pattern	fm.FeatureCircularPattern4	7	Unresolved
get_bounding_box	doc.GetBoundingBox	—	Unresolved
new_part	sw.NewDocument	—	Unreliable

Table 2: MCP server tool status after training and validation.

3.3 Trained System Outputs

Following the completion of tool training and validation described in Section 3.2, the trained MCP server was used to generate a series of parts of increasing complexity. These builds served two purposes: to verify that the confirmed tools functioned correctly when used in combination on real designs, and to explore the practical limits of the system when given open-ended design prompts. For each build, the user provided a natural-language prompt and Claude independently determined the geometry, dimensions, feature sequence, and tool calls required to produce the part. The following subsections present each build in chronological order, along with the prompt used, the tools employed, any issues encountered, and the final result.

3.3.1 Bowl (Shell + Chamfer Test)

During the tool training phase, a simple box was created as a test geometry for validating the newly added shell tool. The user then provided the following prompts:

User Prompt

```
"Can you try and shell out the inside of the block so it is bowl shaped"
```

User Prompt

```
"Try and chamfer all the edges of this bowl"
```

Claude shelled the 50 × 50 × 50 mm solid box by selecting the top face via SelectByRay and calling doc.InsertFeatureShell with a 2 mm wall thickness, removing the top face to create an open bowl shape. All outer edges were then selected in batches using select_by_ray with the append flag set to True , first the top rim edges (outer and inner), then the bottom edges, and finally the vertical corner edges. Each batch was chamfered at 0.5 mm using fm.InsertFeatureChamfer. This however, was not successful.

The shell operation was the first live test of the newly fixed shell tool. The initial server implementation called InsertFeatureShell on the FeatureManager object, which failed because the method only exists on the ModelDoc2 object. After applying the fix described in Section 3.2.4, the shell succeeded on the first attempt. The chamfer did not succeed. The final part contained 2 features: 1 boss extrude and 1 shell, and was saved as Shell-Training.SLDPRT.

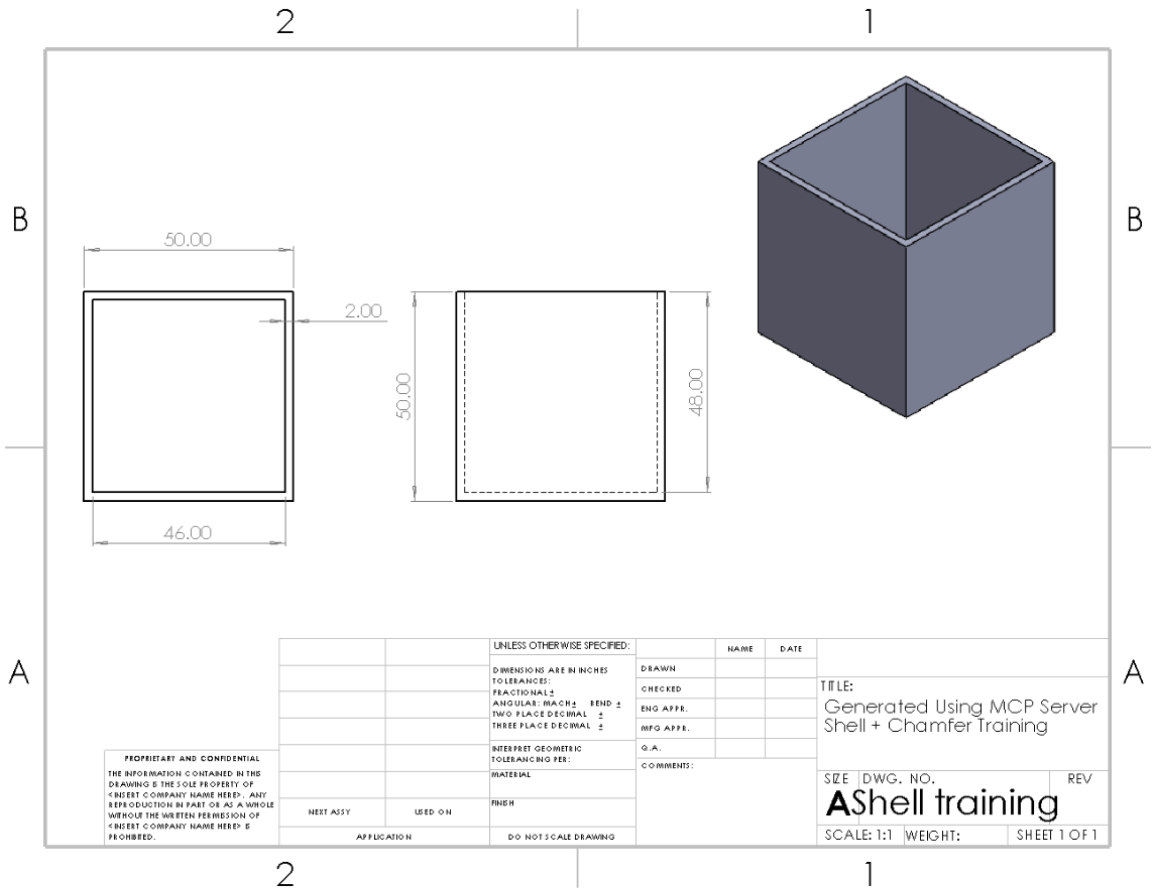


Figure 5: Engineering Drawing of Shell and chamfer training. The bowl was created but, the chamfer failed, likely caused by the thinness of the walls.

3.3.2 Frame Cube

The user provided the following prompt:

User Prompt

```
"I would like you to create me something I can put on my coffee table as a nice display piece the only
requirement is it must be smaller than 265 x 255 x 230 mm you can use any tools you have and if you make
a mistake just undo the mistake and try again I will be away from the keyboard so please try and do this
on your own and work through any issues that may come up."
```

Claude independently designed a wireframe cube sculpture, a 130 mm solid cube with three large rectangular openings cut through all three axes, leaving only the edges as solid 20 mm bars. The design was chosen to create a visually striking conversation piece that resembles an impossible 3D wireframe made from a single piece of metal.

The build used *sketch_rectangle* for the initial cube profile and *extrude_boss* for the solid block. Three rectangular openings were cut through each axis using *sketch_rect_on_face* with world coordinates and *extrude_cut_through_all*. The initial front-to-back cut failed when sketched on the Front reference plane, but succeeded when sketched directly on the model face. This was a recurring pattern observed across multiple builds, reference plane cuts often failed while face-based cuts succeeded, likely due to the cut direction being ambiguous when the sketch plane passes through the middle of existing geometry. Outer corner edges were selected via *select_by_ray* and chamfered at 2 mm using *InsertFeatureChamfer*. An attempt to fillet the inner opening edges failed because the complex geometry at the three-way intersection produced edges that could not accept a fillet at any tested radius. Chrome Stainless Steel was assigned via *SetMaterialPropertyName2*.

The final part contained 5 features: 1 boss extrude, 3 through-all cuts, and 1 chamfer, with a mass of 3.87 kg and overall dimensions of 130 × 130 × 130 mm.

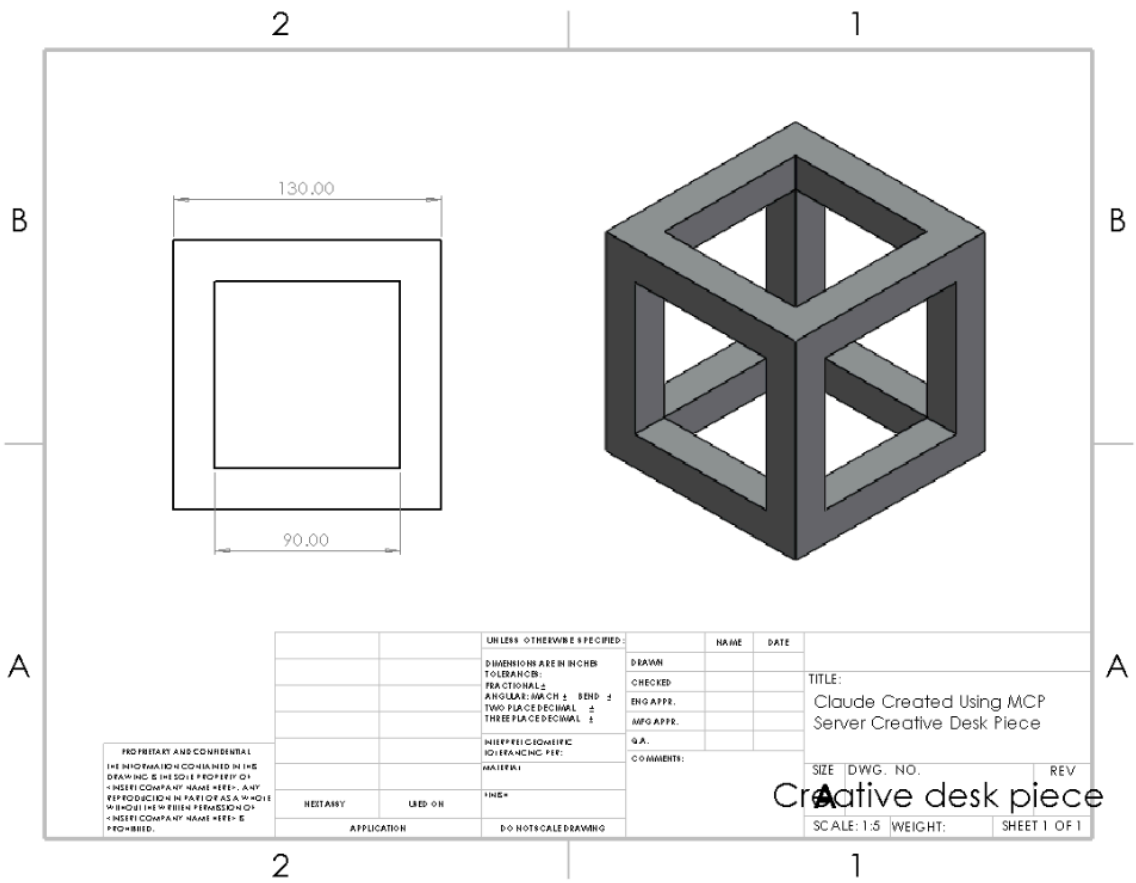


Figure 6: Completed Frame Cube

3.3.3 Geometric Chalice

Following the frame cube, the user requested a more visually dynamic design:

User Prompt

```
"Looks really good now try and make one that is more dynamic and exotic following the same requirements"
```

Claude designed a modernist chalice with a wide flat base (150 mm diameter), a razor-thin stem (20 mm diameter), and a dramatic flared bowl (210 mm diameter). The profile was defined as a 14-point polygon on the Front Plane, with each vertex manually calculated to produce the angular transitions between base, stem, and bowl. The profile was revolved 360° using *fm.FeatureRevolve* with 8 parameters, and the bowl was hollowed using *doc.InsertFeatureShell* with a 4 mm wall thickness and the top face removed.

Six fillets were applied to smooth the transitions: the outer rim (1 mm), inner rim (1 mm), base outer edge (chamfer at 1.5 mm instead of fillet for a sharper look), stem-to-bowl junction (1 mm), base lip (0.5 mm), and bowl bulge point (1 mm). Each edge was selected individually via *select_by_ray* and filleted or chamfered. Two fillet attempts failed at 2 mm and 3 mm radii before succeeding at smaller values, consistent with the minimum-radius constraint imposed by the wall thickness.

An earlier version of the chalice included three decorative through-holes cut into the bowl wall at cardinal positions (front, right, and back). The cut depth was set to 10 mm, which exceeded the 3 mm wall thickness and cut entirely through the bowl in unexpected directions, splitting the body into three separate pieces. Claude identified the issue, deleted all cut features and fillets using *delete_feature*, and rebuilt the decoration-free version. This failure demonstrated a limitation of the blind coordinate approach, without being able to see the model, Claude could not verify that a 10 mm cut depth was inappropriate for a 3 mm wall.

Brass was assigned as the material. The final part contained 9 features: 1 revolve, 1 shell, 5 fillets, 1 chamfer, and 1 material assignment, with a mass of 3.56 kg and overall dimensions of 210 × 195 × 210 mm.

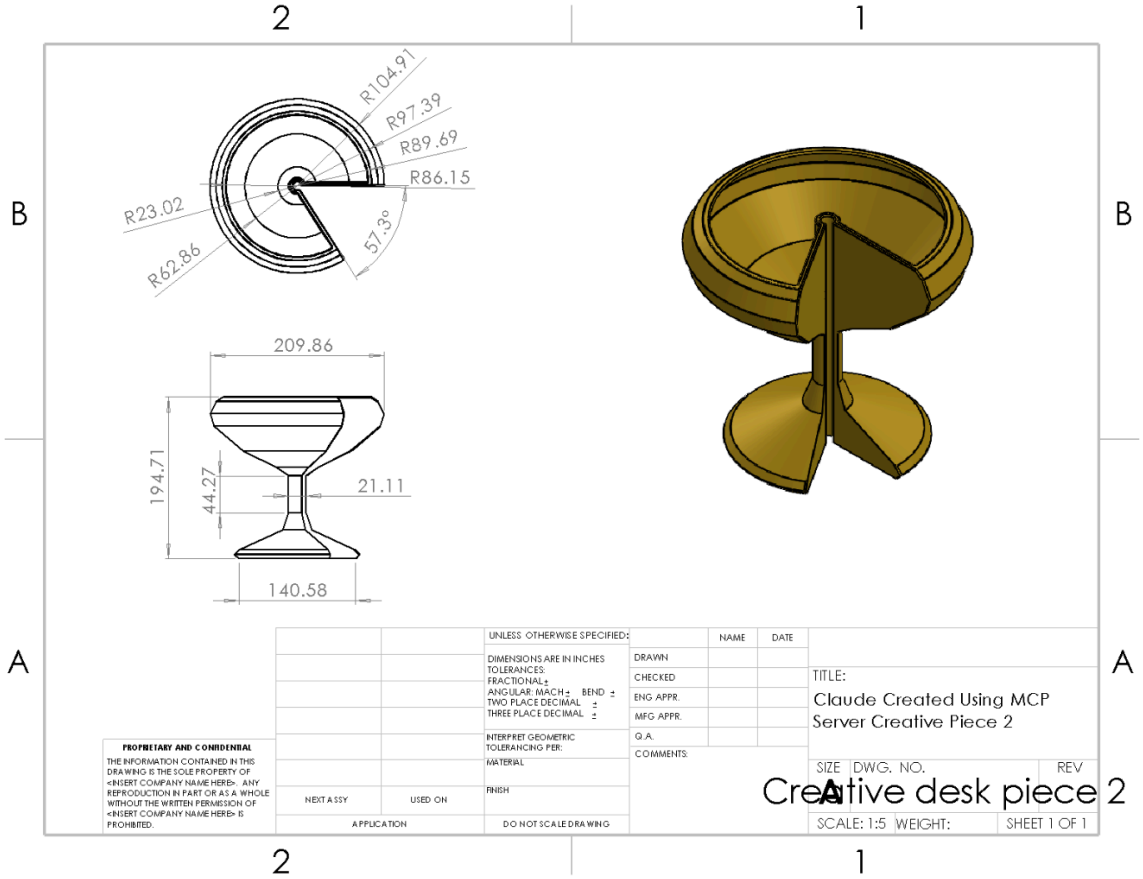


Figure 7: Completed Geometric Chalice. Note the drawing is not fully dimensioned to increase readability

3.3.4 Cessna 172

The user requested a scale model aircraft:

User Prompt

```
"Using all the relevant tools you have please make me a model of a Cessna 172 plane in SolidWorks, don't
worry about scale and sizing I will scale it to a correct size before printing. I have opened a new part
for you to start."
```

This was the most complex build attempted, requiring the widest range of tools and the largest number of features. Claude designed a desk-scale model approximately 200 mm in length.

First Attempt

The fuselage was created as an octagonal cross-section extruded 200 mm, then shaped with 8 taper cuts (4 on the tail, 4 on the nose) using triangular sketches and *extrude_cut_through_all*. Wings were sketched on the Right Plane as a 6-point airfoil profile and

extruded 85 mm outward, with the left wing created via *mirror* about the Right Plane. Horizontal stabilisers, vertical stabiliser, landing gear struts, wheels, propeller spinner (revolved cone), and propeller blades were added as individual features.

The user provided a screenshot of the result, which revealed several features floating disconnected from the fuselage body. The horizontal stabiliser was separated from the tail, the left wing was floating away from the fuselage, and the wheels were detached from the landing gear struts. The root cause was that features extruded from the Right Plane extended in one Z direction only, and when the extrusion direction did not overlap with the fuselage body, the features became separate solid bodies instead of merging with the fuselage. The mirror operation duplicated these separate bodies at the opposite Z offset, compounding the disconnection.

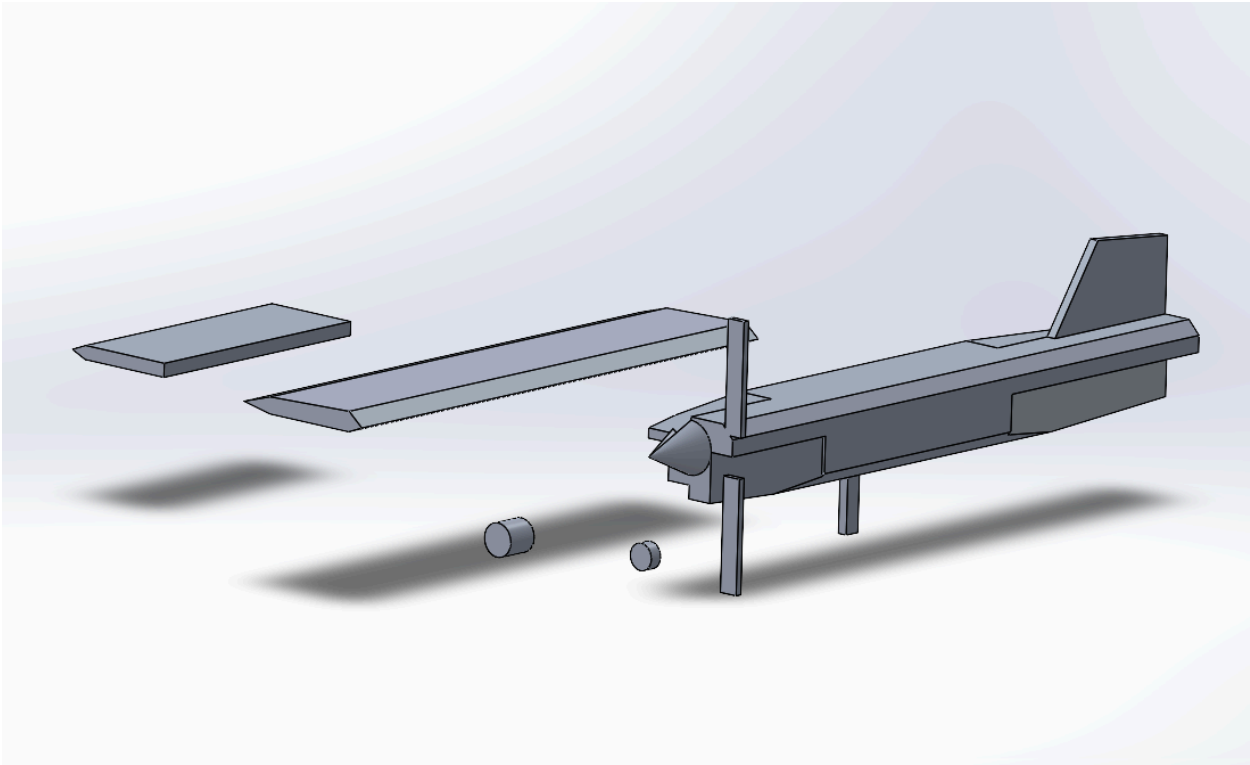


Figure 8: Screenshot that was provided to Claude to show issues with Cessna build

Second Attempt

The user provided a fresh part and Claude rebuilt the model with a corrected strategy:

User Prompt

```
"Why don't you attempt the Cessna one more time I have given you a fresh part opened"
```

The key change was sketching all features on the front plane (Z = 0), which passes through the centre of the fuselage. Since the fuselage extends from Z = −0.015 to Z = +0.015, any feature extruded from Z = 0 inherently overlaps with the fuselage body and merges correctly. Symmetric features (left wing, left stabiliser, left landing gear, left wing strut) were created using *mirror* about the Front Plane rather than the Right Plane.

The fuselage shaping was identical to the first attempt: 8 taper cuts using triangular polygons on the Front and Top planes. The right wing was sketched as a 6-point airfoil cross-section on the Front Plane and extruded 85 mm, then mirrored. The horizontal stabiliser used a 5-point airfoil profile extruded 40 mm and mirrored. The vertical stabiliser was a 4-point trapezoidal profile extruded 3 mm thin on the Front Plane. Landing gear consisted of rectangular struts extruded 3 mm and circular wheels extruded 4–5 mm, with the right-side gear mirrored to create the left side. A propeller spinner was created using *sketch_revolve_profile* with a 3-point triangular profile revolved 360°, and two propeller blades were added as thin extruded rectangles. Diagonal wing struts connecting the wings to the lower fuselage were mirrored.

The final part contained 27 features: 11 boss extrudes, 8 through-all cuts, 4 mirrors, 1 revolve, and associated sketches. 6061 Aluminium Alloy was assigned, giving a mass of 0.467 kg. The tools used spanned nearly the full confirmed tool set: *sketch_polygon*, *sketch_rectangle*, *sketch_circle*, *sketch_revolve_profile*, *extrude_boss*, *extrude_cut_through_all*, *revolve*, *mirror*, *set_material*, and *delete_feature*.

The Cessna build demonstrated both the system's capability and its primary limitation. The range of geometry produced, tapered fuselage, airfoil wings, revolved spinner, mirrored symmetric features, exceeded what either the VBA Macro or Python + COM methods could produce in a single session. However, the disconnected features in the first attempt highlighted the fundamental challenge of building 3D geometry without visual feedback: coordinate errors that would be immediately obvious to a human operator can propagate undetected until the user inspects the result.

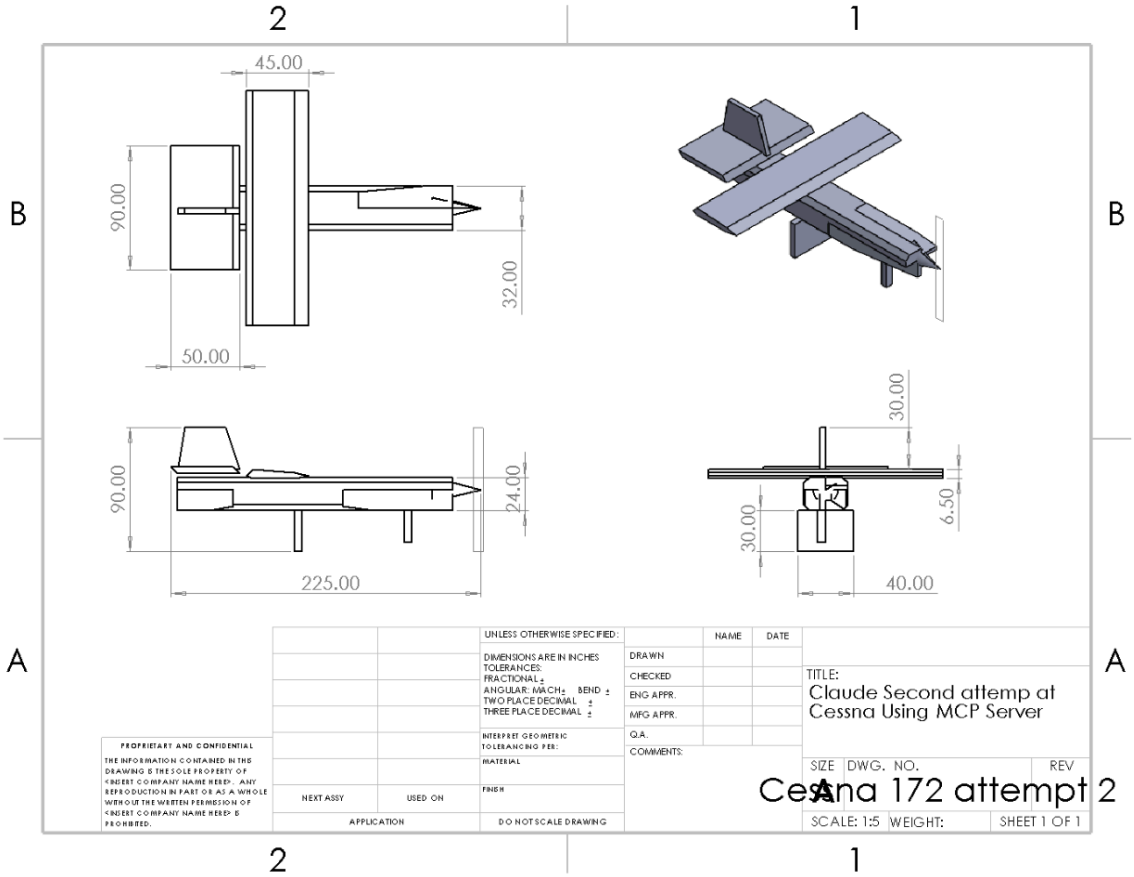


Figure 9: Second attempt at the Cessna the program stopped before the front propeller was created to save credits and time, and the concept had already been demonstrated. Note: the drawing is not fully labeled.

3.3.5 NACA Airfoil - Polygon Method

The user requested a wind tunnel test specimen:

User Prompt

```
"Can you please generate me a section of a NACA 2412 airfoil so that I can test it in a wind tunnel."
```

Claude computed the airfoil coordinates using the standard NACA 4-digit equations: a maximum camber of 2% located at 40% chord, and a maximum thickness of 12%. The mean camber line was calculated using the two-piece parabolic formula, and the thickness distribution was applied perpendicular to the camber line using the standard NACA polynomial coefficients (0.2969, −0.1260, −0.3516, 0.2843, −0.1015). Cosine spacing was used to distribute points along

the chord, concentrating resolution at the leading and trailing edges where curvature is highest. The chord was set to 100 mm and the span to 150 mm, suitable for a small-scale wind tunnel test section.

The first attempt used 81 points (40 per surface plus the leading edge) passed to sketch_polygon on the Front Plane. The extrude_boss call failed, likely because the high vertex count created a sketch that SolidWorks could not process into a valid closed profile. The point count was reduced to 40 (20 per surface), and the polygon was successfully created and extruded 150 mm. However, the user identified that the resulting part had a faceted, angular appearance rather than the smooth curves expected of an airfoil. This was most visible along the upper surface and at the trailing edge, where the shallow convergence angle between the upper and lower surfaces made the straight-line segments between vertices clearly visible.

A second attempt increased the resolution to 160 points (80 per surface) with cosine spacing providing 0.35 mm segment lengths at the leading edge. The polygon was created and extruded successfully. While the higher resolution reduced the visible faceting, the fundamental limitation remained: sketch_polygon connects points with straight line segments, so no matter how many points are used, the result is a polyline approximation rather than a true smooth curve. The trailing edge, where the upper and lower surfaces converge at a very shallow angle, continued to show visible kinks.

The final part contained 2 features: 1 sketch and 1 boss extrude, with a mass of 126 g and a volume of 123.5 cm^3. The part was saved as NACA_2412_Airfoil.SLDPRT.

This build identified a critical gap in the tool set: the absence of a spline tool. The sketch_polygon tool was adequate for prismatic and angular geometry such as the L-bracket and frame cube, but was fundamentally unsuitable for smooth curved profiles. This motivated the development of the sketch_spline tool described in the following section.

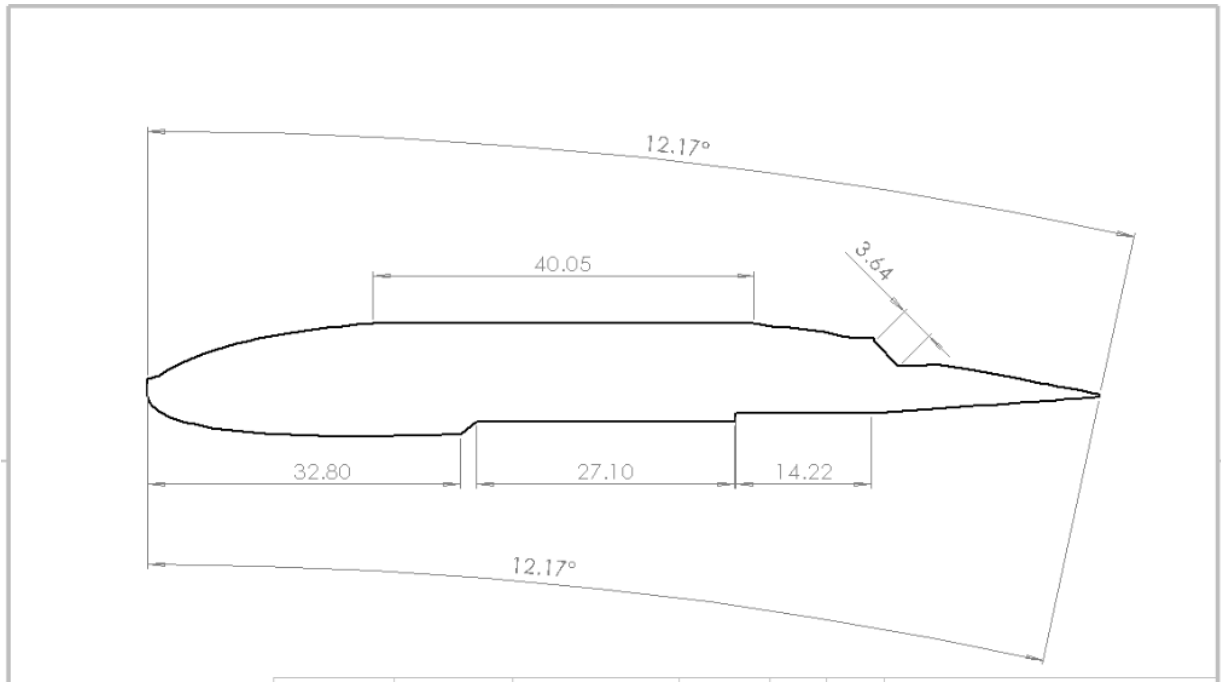


Figure 10: NACA 2412 attempt 1 using 81 points

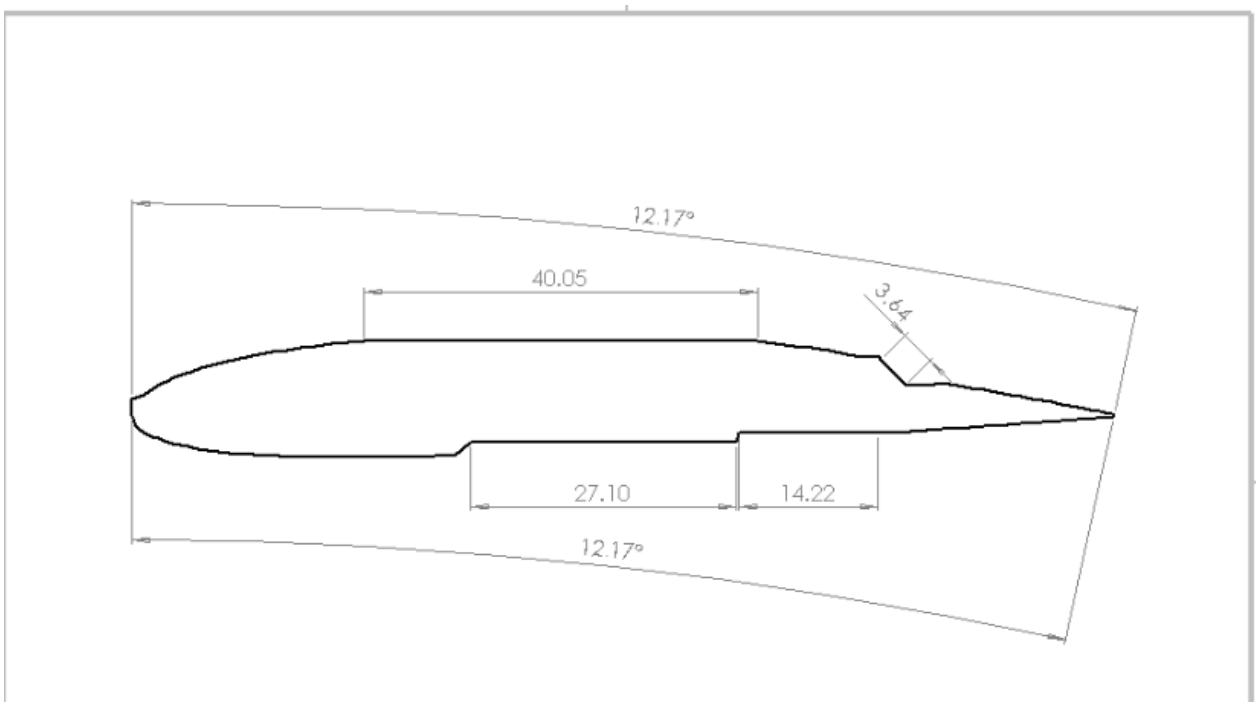


Figure 11: NACA 2412 attempt 2 using 160 Points

3.3.6 NACA Airfoil - Spline Method

Following the identification of the polygon tool's limitations, a new *sketch_spline* tool was developed and added to the MCP server. The tool uses the SolidWorks *SketchManager.CreateSpline2* API method, which takes an array of fit-points and generates an interpolating B-spline curve through them. Unlike *sketch_polygon*, which connects points with straight lines, *sketch_spline* produces a mathematically smooth curve with continuous curvature, the standard method used by engineers when importing airfoil profiles into SolidWorks.

The tool was added to the server using a Python patch script following the same process established during the tool training phase (Section 3.2). The patch inserted a helper function containing the COM interop logic and an async MCP tool wrapper into the existing *solidworks_mcp_server.py* file. Three issues were encountered during integration. First, the patch script inserted the tool code after the *if __name__* guard rather than before it, causing an *IndentationError* that prevented the server from starting. The server was restored from the automatic backup and a corrected patch was applied. Second, the helper function's call to *pythoncom.CoInitialize* was missing the parentheses required to actually invoke the function, meaning COM was never initialised. Third, the async wrapper used a lambda to call the helper function, which bypassed the server's *run_in_com* threading mechanism that handles COM initialisation on the correct thread. Both issues were resolved with single-line edits.

With *sketch_spline* operational, the NACA 2412 airfoil was rebuilt on a fresh part. The same coordinate generation code was used, but with only 60 points (30 per surface), less than half the 160 points required by the polygon method, passed as a closed spline on the Front Plane. The spline was created successfully and extruded 150 mm using *extrude_boss*. ABS was assigned as the material via *set_material*.

The resulting part showed a completely smooth profile with no visible faceting at any point along the surface, including the trailing edge where the polygon method had been most problematic. The leading edge radius was continuous and rounded, and the upper and lower surface curvature transitioned smoothly through the maximum thickness point. The mass and volume (126 g, 123.5 cm^3) were consistent with the polygon version, confirming that the overall geometry was equivalent despite the different construction method.
The final part contained 2 features: 1 spline sketch and 1 boss extrude.

The contrast between the two methods demonstrates the importance of matching the sketch tool to the geometry type. For angular and prismatic parts, *sketch_polygon* and *sketch_rectangle* remain the appropriate tools. For any geometry defined by smooth curves, airfoils, cam profiles, nozzle contours, involute gear teeth, *sketch_spline* is essential. The addition of this single tool substantially expanded the range of engineering geometry the system can produce.

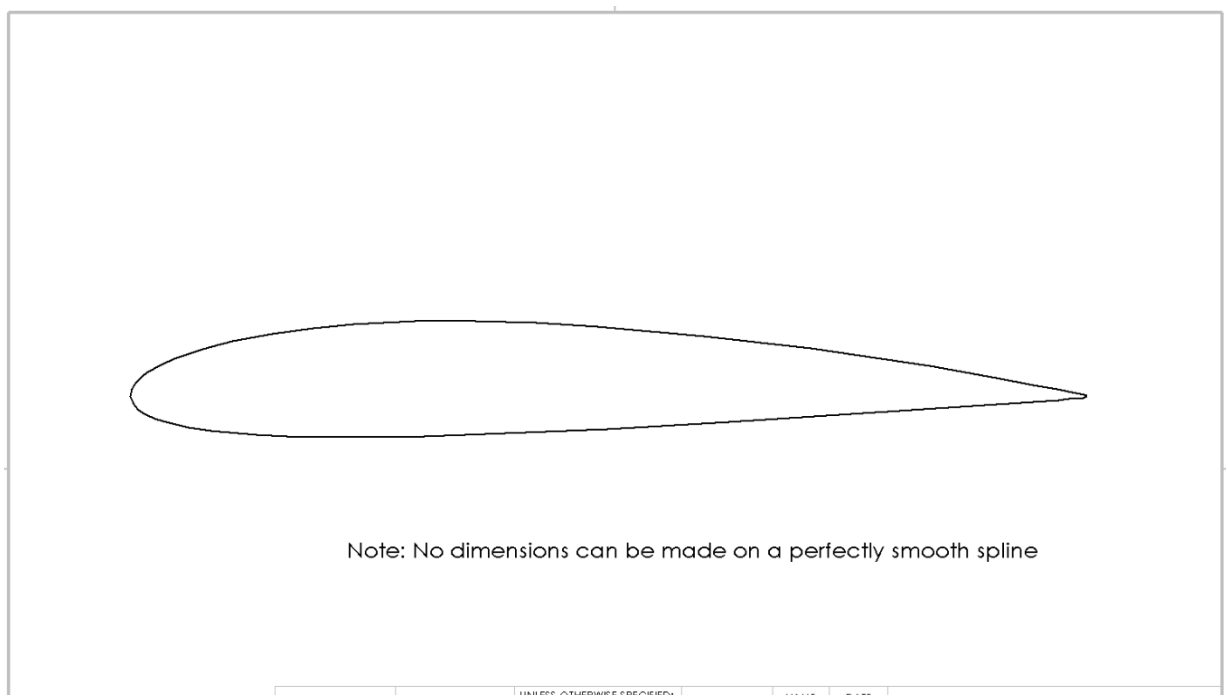


Figure 12: NACA 2412 airfoil created using the Spline tool. This airfoil was perfectly smooth and solved the problems faced by the polygon method

4. Conclusion

This project investigated three methods for enabling Claude to interact with SolidWorks, VBA Macro, Python + COM, and a Local MCP Server, and evaluated how effectively an AI language model can translate natural-language prompts into functional 3D CAD geometry.

The VBA Macro method offered the lowest barrier to entry, requiring no external software and producing the most compact code at 148 lines. However, it was the least geometrically complete of the three methods, failing silently on the vertical plate extrusion and unable to execute any FeatureCut variant through late binding. The Python + COM method produced the correct overall L-bracket shape including fillets and chamfers, but required eleven major revisions across an extended debugging session and ultimately could not create any of the six through-holes due to a parameter count mismatch in FeatureCut4 that was invisible to late-binding dispatch. The Local MCP Server produced the most complete L-bracket, successfully generating the base plate, vertical plate, all six through-holes, and a fillet. Its iterative tool-call architecture, where each operation could be tested, inspected, and retried based on API feedback, proved decisive in resolving the FeatureCut4 parameter count issue that defeated both alternative methods.

Following method selection, the MCP server underwent a systematic training phase that validated and extended its tool set from 25 to 26 tools, with 22 confirmed working through empirical diagnostic testing. The diagnostic methodology developed during this phase, exhaustively testing method names, parameter counts, selection marks, and COM dispatch modes against live SolidWorks geometry, proved to be a reliable technique for discovering correct API calling conventions when official documentation did not match the installed version's behaviour. This approach treated the SolidWorks COM API as a black box and consistently produced working solutions where documentation-driven development had failed.

The trained system demonstrated its capabilities across five builds of increasing complexity: a shelled bowl, a wireframe cube sculpture, a geometric chalice with revolved and shelled features, a multi-feature Cessna 172 model comprising 27 features, and a NACA 2412 airfoil produced in both polygon and spline variants. The airfoil builds directly motivated the addition of the sketch_spline tool, which resolved the fundamental limitation of straight-line polygon approximations for smooth curved geometry. Together, these builds exercised nearly the

full confirmed tool set and produced parts that would be suitable for 3D printing or further engineering development.

The primary limitation identified across all builds was the absence of visual feedback. Claude operates entirely through coordinate-based geometry creation without the ability to see the model, which means that spatial errors such as disconnected features, incorrect cut depths, and mispositioned holes can propagate undetected until the user inspects the result. The Cessna 172's first attempt, where multiple features floated disconnected from the fuselage due to extrusion direction errors, illustrated this limitation clearly. A secondary limitation was the reliance on late-binding COM dispatch, which obscures parameter count mismatches and method signature differences between SolidWorks versions, requiring empirical diagnostic testing rather than straightforward API calls.

Despite these limitations, the results demonstrate that an AI language model can function as an effective CAD operator when given appropriate tooling and an iterative feedback mechanism. The MCP server architecture provides both, and the range of geometry produced, from simple prismatic parts to revolved solids, mirrored assemblies, and spline-based airfoil profiles, substantially exceeds what has been previously demonstrated in AI-assisted CAD workflows.

5. Future Work

The most immediate extension of this project is the addition of visual feedback to the modelling loop. The current system operates blind, relying entirely on coordinate calculations and API return values to infer whether geometry was created correctly. Integrating a screen capture or viewport rendering capability, either through the SolidWorks API's model view export functions or through an external screenshot mechanism, would allow Claude to see the current state of the part after each operation. This single addition would address the root cause of the most significant failures observed during testing, including the disconnected features in the Cessna build and the incorrect cut depths in the chalice's decorative holes.

A second priority is integration with SolidWorks Simulation. Adding MCP tools that interface with the Simulation API would allow the system to apply loads and boundary conditions, run static stress analyses, and read back von Mises stress distributions. With this feedback loop, the AI could move beyond executing explicit user instructions and begin performing design optimisation autonomously, removing material from low-stress regions, re-analysing, and verifying that stress remains within acceptable limits. This would transform the system from a CAD operator into a design assistant capable of engineering reasoning.

The current tool set also has specific gaps that limited the complexity of parts produced during testing. The circular pattern tool, which was implemented but remained unresolved at the end of the training phase, would enable radial hole patterns and rotational symmetry features common in flanges, gears, and turbine components. A sweep tool, which creates geometry by moving a profile along a path, would enable piping, conduit, and complex curved structural members. A loft tool, which blends between two or more profiles at different positions, would allow the creation of aerodynamic fairings, bottle shapes, and transition sections, geometry types that cannot be produced with any combination of the current extrude and revolve tools.

Addressing the COM dispatch reliability issues identified throughout this project would also improve the system's robustness. The _FlagAsMethod workaround, the empirical parameter count discovery process, and the need to re-select sketches between failed attempts all stem from limitations of late-binding COM dispatch in pywin32. Migrating to early binding using generated type libraries, or implementing a thin C++ or C# COM wrapper that exposes a simplified and version-stable interface, would eliminate an entire class of errors that currently require manual diagnostic intervention.

Finally, the diagnostic methodology developed during the training phase could itself be automated. The four-stage test-diagnose-patch-verify process was applied manually to each tool, with diagnostic scripts written by Claude and executed by the user. Embedding this diagnostic capability directly into the MCP server, allowing it to detect a failed API call, generate and execute a parameter sweep, identify the working combination, and update its own calling convention, would make the system self-healing and reduce the need for human intervention when deploying on new SolidWorks versions or service packs.

6. References

[1] Dassault Systèmes, "SOLIDWORKS: 3D CAD Design Software," Dassault Systèmes. [Online]. Available: https://www.3ds.com/products/solidworks. [Accessed: Apr. 23, 2026].

[2] Anthropic, "Intro to Claude," Claude API Documentation. [Online]. Available: https://platform.claude.com/docs/en/intro. [Accessed: Apr. 23, 2026].

[3] PTC, "Onshape: Cloud SaaS CAD Solution," PTC. [Online]. Available: https://www.ptc.com/en/products/onshape. [Accessed: Apr. 23, 2026].

[4] Anthropic, "Introducing the Model Context Protocol," Anthropic, Nov. 2024. [Online]. Available: https://www.anthropic.com/news/model-context-protocol. [Accessed: Apr. 23, 2026].

[5] M. Hammond, "pywin32: Python for Win32 Extensions," Python Package Index. [Online]. Available: https://pypi.org/project/pywin32/. [Accessed: Apr. 23, 2026].

[6] Model Context Protocol, "Specification," The Linux Foundation, Nov. 2025. [Online]. Available: https://modelcontextprotocol.io/specification/2025-11-25. [Accessed: Apr. 23, 2026].

[7] Dassault Systèmes, "SOLIDWORKS API Help," SolidWorks Online Documentation. [Online]. Available: https://help.solidworks.com/. [Accessed: Apr. 23, 2026].

[8] Dave McG., "L Bracket Small," MyMiniFactory. [Online]. Available: https://www.myminifactory.com/object/3d-print-l-bracket-small-103067. [Accessed: Apr. 23, 2026].

[9] medyashkinvl, "Parametric L-Bracket Mount M3-M5 3D Printable STL," Cults3D, Dec. 2025. [Online]. Available: https://cults3d.com/en/3d-model/tool/parametric-l-bracket-mount-m3-m5-3d-printable-stl. [Accessed: Apr. 23, 2026].

Appendices

Appendix A

The following presents the complete VBA macro used to automate L-bracket generation in SolidWorks 2024 via the built-in VBA editor. The macro was 148 lines and is presented as-is, directly executable by opening a blank part and running via Tools → Macro → Run.

```
' ============================================================
' SolidWorks 2024 VBA Macro - Fully Parametric L-Bracket
' ============================================================
' 1. In SolidWorks: File > New > Part > OK
' 2. Tools > Macro > Run > select this file
' ============================================================

Sub Main()

    Dim swApp As Object
    Dim swModel As Object
    Dim swFeatMgr As Object
    Dim swSketchMgr As Object
    Dim swFeat As Object
    Dim eq As Object

    ' --- Dimensions in metres (SolidWorks API standard) ---
    Dim BL As Double: BL = 0.12       ' Base Length 120mm
    Dim BW As Double: BW = 0.06       ' Base Width 60mm
    Dim TH As Double: TH = 0.008      ' Thickness 8mm
    Dim VH As Double: VH = 0.08       ' Vert Height 80mm
    Dim BHR As Double: BHR = 0.005    ' Base Hole Radius 5mm (Dia 10)
    Dim VHR As Double: VHR = 0.004    ' Vert Hole Radius 4mm (Dia 8)
    Dim RT As Double: RT = 0.006      ' Rib Thickness 6mm
    Dim FR As Double: FR = 0.005      ' Fillet Radius 5mm
    Dim CD As Double: CD = 0.001      ' Chamfer Distance 1mm
    Dim HM As Double: HM = 0.015     ' Hole margin from edge 15mm

    ' --- Connect ---
    Set swApp = GetObject(, "SldWorks.Application")
    Set swModel = swApp.ActiveDoc
    If swModel Is Nothing Then
        MsgBox "Open a blank Part first (File > New > Part > OK)."
        Exit Sub
    End If
    Set swFeatMgr = swModel.FeatureManager
    Set swSketchMgr = swModel.SketchManager


    ' ============================================================
    ' BASE PLATE - rectangle + 4 mounting holes in one sketch
    ' ============================================================
    swModel.Extension.SelectByID2 "Top Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0
    swSketchMgr.InsertSketch True

    swSketchMgr.CreateCornerRectangle -BL / 2, 0, 0, BL / 2, BW, 0

    swSketchMgr.CreateCircleByRadius -BL / 2 + HM, HM, 0, BHR
    swSketchMgr.CreateCircleByRadius BL / 2 - HM, HM, 0, BHR
    swSketchMgr.CreateCircleByRadius -BL / 2 + HM, BW - HM, 0, BHR
    swSketchMgr.CreateCircleByRadius BL / 2 - HM, BW - HM, 0, BHR

    swSketchMgr.InsertSketch True

    Set swApp = GetObject(, "SldWorks.Application")
    Set swModel = swApp.ActiveDoc
```

```
    Set swFeatMgr = swModel.FeatureManager
    Set swSketchMgr = swModel.SketchManager

    Set swFeat = swFeatMgr.FeatureExtrusion3(True, False, False, 0, 0, TH, 0, False, False, False,
False, 0, 0, False, False, False, False, True, True, True, 0, 0, False)
    If Not swFeat Is Nothing Then swFeat.Name = "Base Plate"


    ' ==========================================================
    ' VERTICAL PLATE - rectangle + 2 holes in one sketch
    ' ==========================================================
    Set swApp = GetObject(, "SldWorks.Application")
    Set swModel = swApp.ActiveDoc
    Set swFeatMgr = swModel.FeatureManager
    Set swSketchMgr = swModel.SketchManager

    swModel.Extension.SelectByID2 "Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0
    swSketchMgr.InsertSketch True

    swSketchMgr.CreateCornerRectangle -BL / 2, 0, 0, -BL / 2 + TH, VH, 0

    Dim hcx As Double: hcx = -BL / 2 + TH / 2
    swSketchMgr.CreateCircleByRadius hcx, VH / 3, 0, VHR
    swSketchMgr.CreateCircleByRadius hcx, 2 * VH / 3, 0, VHR

    swSketchMgr.InsertSketch True

    Set swApp = GetObject(, "SldWorks.Application")
    Set swModel = swApp.ActiveDoc
    Set swFeatMgr = swModel.FeatureManager
    Set swSketchMgr = swModel.SketchManager

    Set swFeat = swFeatMgr.FeatureExtrusion3(True, False, False, 6, 0, BW, 0, False, False, False,
False, 0, 0, False, False, False, False, True, True, True, 0, 0, False)
    If Not swFeat Is Nothing Then swFeat.Name = "Vertical Plate"


    ' ==========================================================
    ' REINFORCING RIB - triangular gusset
    ' ==========================================================
    Set swApp = GetObject(, "SldWorks.Application")
    Set swModel = swApp.ActiveDoc
    Set swFeatMgr = swModel.FeatureManager
    Set swSketchMgr = swModel.SketchManager

    swModel.Extension.SelectByID2 "Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0
    swSketchMgr.InsertSketch True

    Dim rx As Double: rx = -BL / 2 + TH
    swSketchMgr.CreateLine rx, 0, 0, rx + 0.04, 0, 0
    swSketchMgr.CreateLine rx + 0.04, 0, 0, rx, 0.04, 0
    swSketchMgr.CreateLine rx, 0.04, 0, rx, 0, 0

    swSketchMgr.InsertSketch True

    Set swApp = GetObject(, "SldWorks.Application")
    Set swModel = swApp.ActiveDoc
    Set swFeatMgr = swModel.FeatureManager
    Set swSketchMgr = swModel.SketchManager

    Set swFeat = swFeatMgr.FeatureExtrusion3(True, False, False, 6, 0, RT, 0, False, False, False,
False, 0, 0, False, False, False, False, True, True, True, 0, 0, False)
    If Not swFeat Is Nothing Then swFeat.Name = "Reinforcing Rib"


    ' ==========================================================
    ' FILLETS - 5mm on internal corner
```

```
    ' ==========================================================
    Set swApp = GetObject(, "SldWorks.Application")
    Set swModel = swApp.ActiveDoc
    Set swFeatMgr = swModel.FeatureManager

    swModel.ClearSelection2 True
    On Error Resume Next
    Dim bFillet As Boolean
    bFillet = swModel.Extension.SelectByRay(-BL / 2 + TH, 0.001, BW / 2, -1, -1, 0, 0.003, 1, False, 0,
0)
    If bFillet Then
        Set swFeat = swFeatMgr.FeatureFillet3(194, FR, 0, 0, 0, 0, 0, Empty, Empty, Empty, Empty, Empty,
Empty, Empty)
        If Not swFeat Is Nothing Then swFeat.Name = "Internal Fillet"
    End If
    On Error GoTo 0


    ' ==========================================================
    ' CHAMFERS - 1mm on external edges
    ' ==========================================================
    Set swApp = GetObject(, "SldWorks.Application")
    Set swModel = swApp.ActiveDoc
    Set swFeatMgr = swModel.FeatureManager

    swModel.ClearSelection2 True
    On Error Resume Next
    Dim bCham As Boolean
    bCham = swModel.Extension.SelectByRay(-BL / 2 + TH / 2, VH, BW / 2, 0, 1, 0, 0.003, 1, False, 0, 0)
    bCham = swModel.Extension.SelectByRay(0, -TH, 0, 0, -1, 0, 0.003, 1, True, 0, 0)
    If bCham Then
        Set swFeat = swFeatMgr.InsertFeatureChamfer(4, 1, CD, 0, 0, 0, 0, 0)
        If Not swFeat Is Nothing Then swFeat.Name = "External Chamfers"
    End If
    On Error GoTo 0


    ' ==========================================================
    ' MATERIAL - Aluminum 6061
    ' ==========================================================
    Set swApp = GetObject(, "SldWorks.Application")
    Set swModel = swApp.ActiveDoc

    On Error Resume Next
    swModel.SetMaterialPropertyName2 "", "solidworks materials.sldmat", "6061 Alloy"
    swModel.SetMaterialPropertyName2 "", "SolidWorks Materials.sldmat", "6061 Alloy"
    swModel.SetMaterialPropertyName2 "", "SOLIDWORKS Materials.sldmat", "6061 Alloy"
    On Error GoTo 0


    ' ==========================================================
    ' EQUATIONS - Global variables for parametric control
    ' ==========================================================
    Set swApp = GetObject(, "SldWorks.Application")
    Set swModel = swApp.ActiveDoc

    On Error Resume Next
    Set eq = swModel.GetEquationMgr
    If Not eq Is Nothing Then
        eq.Add2 -1, """Thickness"" = 8mm", True
        eq.Add2 -1, """BaseLength"" = 120mm", True
        eq.Add2 -1, """BaseWidth"" = 60mm", True
        eq.Add2 -1, """VertHeight"" = 80mm", True
        eq.Add2 -1, """BaseHoleDia"" = 10mm", True
        eq.Add2 -1, """VertHoleDia"" = 8mm", True
        eq.Add2 -1, """RibThickness"" = 6mm", True
        eq.Add2 -1, """FilletRadius"" = 5mm", True
```

```
        eq.Add2 -1, """ChamferDist"" = 1mm", True
        eq.EvaluateAll
    End If
    On Error GoTo 0


    ' ==========================================================
    ' REBUILD AND ZOOM
    ' ==========================================================
    Set swApp = GetObject(, "SldWorks.Application")
    Set swModel = swApp.ActiveDoc
    swModel.ForceRebuild3 True
    swModel.ViewZoomtofit2

    MsgBox "L-Bracket complete!"

End Sub
```

Appendix B

The following presents the critical sections of the Python script used to automate L-bracket generation in SolidWorks 2024 via the COM API. The full script was 749 lines.

B.1 Imports and Unit Conversion

```
import win32com.client
import pythoncom
import time, os, sys

MM = 1.0e-3
def v(mm):
    """Convert millimetres to metres (SolidWorks internal unit)."""
    return float(mm) * MM
```

B.2 Design Parameters

All key dimensions are stored in a single dictionary and later injected into the SolidWorks Equations Manager as global variables.

```
P = {
    "BaseLength":       120.0,   # mm
    "BaseWidth":         60.0,
    "Thickness":          8.0,
    "VerticalHeight":    80.0,
    "BaseMountHoleDia":  10.0,
    "VertHoleDia":        8.0,
    "RibThickness":       6.0,
    "FilletRadius":       5.0,
    "ChamferSize":        1.5,
    "HoleEdgeOffset":    20.0,
    "VertHoleSpacing":   40.0,
}
```

B.3 COM Connection

The script iterates through version-specific ProgIDs and pre-flags ambiguous method names on the application object.

```
def connect_solidworks():
    sw = None
    for pid in ["SldWorks.Application",
                "SldWorks.Application.31",    # SolidWorks 2023
                "SldWorks.Application.30",    # SolidWorks 2022
                "SldWorks.Application.32"]:   # SolidWorks 2024
        try:
            sw = win32com.client.Dispatch(pid)
            if sw:
                break
        except Exception:
            continue
    sw.Visible = True
    sw.UserControl = True
    for m in ["CloseDoc", "OpenDoc6", "NewDocument",
              "GetUserPreferenceStringValue", "RunMacro2"]:
        try:
```

```
            sw._FlagAsMethod(m)
        except Exception:
            pass
    return sw
```

B.4 Template Discovery

A three-tier search strategy locates the part template file across SolidWorks installations.

```
def find_template(sw, ext=".prtdot"):
    # Tier 1: Query SolidWorks preferences
    try:
        path = sw.GetUserPreferenceStringValue(7)
        if path and os.path.isfile(path):
            return path
    except Exception:
        pass
    # Tier 2: Scan version-specific directories (2018-2026)
    for year in range(2026, 2017, -1):
        for base in [r"C:\ProgramData\SolidWorks\SOLIDWORKS",
                     r"C:\ProgramData\SOLIDWORKS\SOLIDWORKS"]:
            d = f"{base} {year}\\templates"
            if os.path.isdir(d):
                for f in os.listdir(d):
                    if f.lower().endswith(ext.lower()):
                        return os.path.join(d, f)
    # Tier 3: Recursive filesystem search (omitted for brevity)
    return None
```

B.5 Equation Injection with Cascading Fallback

Global variables are added to the Equations Manager, attempting multiple method signatures to accommodate version differences.

```
def add_equations(model):
    eq = model.GetEquationMgr()
    for m in ["Add3", "Add2", "Add"]:
        eq._FlagAsMethod(m)
    idx = 0
    for name, val in P.items():
        s = f'"{name}" = {val}'
        for method in ["Add3", "Add2", "Add"]:
            try:
                if method == "Add3":
                    eq.Add3(idx, s, True, "")
                elif method == "Add2":
                    eq.Add2(idx, s, True)
                else:
                    eq.Add(idx, s)
                break
            except Exception:
                continue
        idx += 1
    model.EditRebuild3()
```

B.6 Boss-Extrude Operation

The *_FlagAsMethod* call is made on the same *FeatureManager* wrapper used for the subsequent method call, which is critical for late-binding dispatch.

```
def extrude_boss(model, depth_mm, mid_plane=False):
    fm = model.FeatureManager
    fm._FlagAsMethod("FeatureExtrusion2")
    end_cond = 6 if mid_plane else 0   # swEndCondMidPlane or swEndCondBlind
    fm.FeatureExtrusion2(
        not mid_plane, False, False,
        end_cond, end_cond,
        v(depth_mm), v(depth_mm) if mid_plane else 0,
        False, False, False, False, 0, 0,
        False, False, False, False,
        True, True, True, 0, 0, False)
```

B.7 Cut-Extrude with InvokeTypes

This is the most technically complex section of the script. The *InvokeTypes* method bypasses all late-binding dispatch abstraction by specifying explicit COM type constants for every parameter.

```
VT_BOOL = 11; VT_I4 = 3; VT_R8 = 5; VT_DISPATCH = 9

def cut_through_all(model, sw):
    fm = model.FeatureManager
    oleobj = fm._oleobj_

    # Resolve method dispatch ID by name
    dispid = oleobj.GetIDsOfNames(0, 'FeatureCut4')
    if isinstance(dispid, tuple):
        dispid = dispid[0]

    # Define explicit types for all 24 parameters
    arg_types = (
        (VT_BOOL, 1),    #  1 Sd (single direction)
        (VT_BOOL, 1),    #  2 Flip
        (VT_BOOL, 1),    #  3 Dir2
        (VT_I4,   1),    #  4 T1 (end condition: 1 = ThroughAll)
        (VT_I4,   1),    #  5 T2
        (VT_R8,   1),    #  6 D1 (depth)
        (VT_R8,   1),    #  7 D2
        (VT_BOOL, 1),    #  8 DraftOnOff1
        (VT_BOOL, 1),    #  9 DraftOnOff2
        (VT_BOOL, 1),    # 10 DraftOutward1
        (VT_BOOL, 1),    # 11 DraftOutward2
        (VT_R8,   1),    # 12 DraftAngle1
        (VT_R8,   1),    # 13 DraftAngle2
        (VT_BOOL, 1),    # 14 OffsetReverse1
        (VT_BOOL, 1),    # 15 OffsetReverse2
        (VT_BOOL, 1),    # 16 TranslateSurface1
        (VT_BOOL, 1),    # 17 TranslateSurface2
        (VT_BOOL, 1),    # 18 NormalCut
        (VT_BOOL, 1),    # 19 UseFeatScope
        (VT_BOOL, 1),    # 20 UseAutoSelect
        (VT_BOOL, 1),    # 21 AssemblyFeatureScope
        (VT_BOOL, 1),    # 22 AutoDetermineAssemblyFeatScope
        (VT_BOOL, 1),    # 23 AssemblyFeatureFlip
        (VT_I4,   1),    # 24 T0 (start condition: 0 = sketch plane)
    )
```

```
    result = oleobj.InvokeTypes(
        dispid, 0, pythoncom.DISPATCH_METHOD,
        (VT_DISPATCH, 1),   # Return type: IFeature
        arg_types,
        True, False, False, 1, 0, 0.01, 0.01,
        False, False, False, False, 0.0, 0.0,
        False, False, False, False,
        True, True, True, False, False, False, 0
    )
    return result
```

If *InvokeTypes* fails, the script cascades through *_FlagAsMethod*-based calls to *FeatureCut4* (24 parameters), *FeatureCut3* (20 parameters), *FeatureCut2* (18 parameters), and *FeatureCut* (13 parameters). As a final fallback, a VBA macro is written to disk and executed via *RunMacro2*.

B.8 Geometry Creation Example — Base Plate

Each feature follows the same workflow: select plane, open sketch, create geometry, dimension, close sketch, and extrude.

```
def create_base_plate(model):
    L, W, T = P["BaseLength"], P["BaseWidth"], P["Thickness"]
    sel(model, "Top Plane", "PLANE")
    model.SketchManager.InsertSketch(True)
    model.SketchManager.CreateCenterRectangle(0, 0, 0, v(L/2), v(W/2), 0)
    model.AddDimension2(0, v(W/2 + 5), 0)
    model.AddDimension2(v(L/2 + 5), 0, 0)
    model.SketchManager.InsertSketch(True)
    extrude_boss(model, T)
    rename_last(model, "Base Plate")
    model.EditRebuild3()
```

B.9 Material Assignment

Multiple database and material name combinations are attempted to accommodate locale and version differences.

```
def assign_material(model):
    for db in ["solidworks materials.sldmat", "SOLIDWORKS Materials"]:
        for mat in ["6061 Alloy", "6061-T6 (SS)", "6061-T6"]:
            try:
                model.SetMaterialPropertyName2("", db, mat)
                return
            except Exception:
                continue
```

Appendix C

C.1 Claude Desktop Configuration

The following JSON configuration was placed in the Claude Desktop application configuration file, accessed via Settings → Developer → Edit Config. The command field specifies the full path to the Python interpreter and the args field specifies the path to the server script.

```
{
  "mcpServers": {
    "solidworks": {
      "command": "C:\\Users\\lfawc\\AppData\\Local\\Python\\pythoncore-3.14-64\\python.exe",
      "args": ["C:\\SolidWorksMCP\\solidworks_mcp_server.py"],
      "env": {
        "PYTHONUNBUFFERED": "1"
      }
    }
  }
}
```

C.2 MCP Server Script (*Solidworks_mcp_server.py*)

The following presents the critical sections of the Python MCP server script used to bridge Claude Desktop to SolidWorks 2024 via the COM API. The full script was 458 lines and exposed 25 remotely callable tools.

C.2.1 Imports and COM Initialisation

```
import json
import logging
import asyncio
import pythoncom
import win32com.client
from mcp.server.fastmcp import FastMCP

_sw_app = None

def _ensure_com():
    global _sw_app
    pythoncom.CoInitialize()
    if _sw_app is None:
        _sw_app = win32com.client.Dispatch("SldWorks.Application")
        _sw_app.Visible = True
    return _sw_app
```

Unlike the Python + COM method, which iterates through version-specific *ProgIDs*, the MCP server relies on the generic *SldWorks.Application ProgID* and requires SolidWorks to be running before the server process starts. The *pythoncom.CoInitialize()* call is made on every entry point because COM calls execute on background threads via *asyncio.run_in_executor()*.

C.2.2 COM Type Helpers

All SolidWorks API methods that accept optional or output parameters require explicitly typed COM variants. Two helper functions construct these variants for reuse across the script.

```
def _null():
    return win32com.client.VARIANT(pythoncom.VT_DISPATCH, None)

def _byref_int():
    return win32com.client.VARIANT(pythoncom.VT_BYREF | pythoncom.VT_I4, 0)
```

The *_null()* variant is required for the callout parameter of *SelectByID2*, which raises a type-mismatch error if Python's None is passed directly. The *_byref_int()* variant is used for error and warning output parameters on methods such as *OpenDoc6* and *SaveAs3*.

C.2.3 Async-to-COM Bridge

Each MCP tool is an async coroutine, but SolidWorks COM calls are blocking. The bridge function offloads every COM call to the default thread-pool executor.

```
async def run_in_com(func, *args, **kwargs):
    loop = asyncio.get_running_loop()
    return await loop.run_in_executor(None, lambda: func(*args, **kwargs))
```

C.2.4 Sketch Selection

SolidWorks 2024 requires explicit sketch selection before extrusion. The server walks the feature tree to find the last *ProfileFeature* entry and selects it by name.

```
def _find_last_sketch_name():
    _, doc = _get_active_doc()
    last_sketch = None
    feat = doc.FirstFeature
    while feat is not None:
        try:
            if feat.GetTypeName2 == "ProfileFeature":
                last_sketch = feat.Name
        except Exception:
            pass
        try:
            feat = feat.GetNextFeature
        except Exception:
            break
    return last_sketch

def _select_last_sketch():
    _, doc = _get_active_doc()
    name = _find_last_sketch_name()
    if name is None:
        raise RuntimeError("No sketch found in the feature tree.")
    nv = _null()
    doc.Extension.SelectByID2(name, "SKETCH", 0, 0, 0, False, 0, nv, 0)
    return name
```

C.2.5 World-to-Sketch Coordinate Transform

Face-based sketching requires converting world coordinates to the face's local sketch coordinate system. The transform matrix is read from the active sketch and applied via manual matrix multiplication, bypassing *MathUtility.CreatePoint* which fails due to COM type-marshalling issues.

```
def _world_to_sketch_coords(doc, wx, wy, wz):
    try:
        sketch = doc.SketchManager.ActiveSketch
        if sketch is not None:
            xform = sketch.ModelToSketchTransform
            if xform is not None:
                m = xform.ArrayData
                a,b,c = float(m[0]),float(m[1]),float(m[2])
                d,e,f = float(m[3]),float(m[4]),float(m[5])
                g,h,i = float(m[6]),float(m[7]),float(m[8])
                tx,ty,tz = float(m[9]),float(m[10]),float(m[11])
                sx = a*wx + d*wy + g*wz + tx
                sy = b*wx + e*wy + h*wz + ty
                return sx, sy
    except Exception:
        pass
    return wx, wy
```

The 4×4 transformation matrix stored in ArrayData contains rotation components (indices 0–8), translation components (indices 9–11), and a scale factor (index 12). Only the first two rows of the rotation-translation product are needed to obtain the 2D sketch coordinates.

C.2.6 Sketching on Model Faces

Holes and cut profiles are drawn on model faces rather than reference planes. The face is selected by ray, a sketch is opened on it, world coordinates are transformed to sketch coordinates, and the sketch is closed.

```
def _sketch_circle_on_face(face_x, face_y, face_z,
                           ray_dx, ray_dy, ray_dz,
                           world_cx, world_cy, world_cz, radius):
    _, doc = _get_active_doc()
    doc.ClearSelection2(True)
    ok = doc.Extension.SelectByRay(
        face_x, face_y, face_z,
        ray_dx, ray_dy, ray_dz,
        0.001, 2, False, 0, 0)
    if not ok:
        raise RuntimeError("Could not select face at given position")
    sm = doc.SketchManager
    sm.InsertSketch(True)
    sx, sy = _world_to_sketch_coords(doc, world_cx, world_cy, world_cz)
    sm.CreateCircleByRadius(sx, sy, 0, radius)
    sm.InsertSketch(True)
    doc.ClearSelection2(True)
    return {"status": "ok"}
```

The *SelectByRay* parameters specify the ray origin, direction, tolerance radius (0.001 m), entity type (2 = face), and selection flags. The ray origin is placed slightly outside the target face and directed inward.

C.2.7 Boss-Extrude Operation

The *FeatureExtrusion2* method with 20 parameters was confirmed working on SolidWorks 2024 SP5. A fallback to *FeatureExtrusion3* (23 parameters) is provided for version compatibility.

```
def _extrude_boss(depth, reverse=False):
    sw, doc = _get_active_doc()
    _select_last_sketch()
    fm = doc.FeatureManager
    feat = None
    attempts = [
        ("FeatureExtrusion2", [True, reverse, False, 0, 0, depth, 0.0,
         False, False, False, False, 0.0, 0.0, False, False, False, False,
         True, False, True]),
        ("FeatureExtrusion3", [True, reverse, False, 0, 0, depth, 0.0,
         False, False, False, False, 0.0, 0.0, False, False, False, False,
         True, False, True, 0, 0.0, False]),
    ]
    for name, params in attempts:
        try:
            feat = getattr(fm, name)(*params)
            if feat is not None:
                break
        except Exception:
            continue
    if feat is None:
        raise RuntimeError("Extrude failed.")
    return {"status": "ok", "message": f"Extruded boss by {depth}m"}
```

C.2.8 Cut-Extrude Operation

The *FeatureCut4* method with exactly 27 parameters was determined empirically as the only working cut method on SolidWorks 2024 SP5. This was established by running a diagnostic script that tested all known *FeatureCut* variants (*FeatureCut* through *FeatureCut4*) with parameter counts ranging from 13 to 28. The sketch is re-selected immediately before each attempt, as failed COM calls clear the active selection.

```
def _extrude_cut(depth, reverse=False):
    sw, doc = _get_active_doc()
    sketch_name = _find_last_sketch_name()
    if sketch_name is None:
        raise RuntimeError("No sketch found.")
    fm = doc.FeatureManager
    feat = None
    nv = _null()
    attempts = [
        [True, reverse, False, 0, 0, depth, 0.0,
         False, False, False, False, 0.0, 0.0,
         False, False, False, False, False,
         True, True, True, True, False, 0, 0.0, False, False],
        [True, not reverse, False, 0, 0, depth, 0.0,
         False, False, False, False, 0.0, 0.0,
         False, False, False, False, False,
         True, True, True, True, False, 0, 0.0, False, False],
    ]
    for params in attempts:
        try:
            doc.Extension.SelectByID2(
                sketch_name, "SKETCH", 0, 0, 0, False, 0, nv, 0)
            feat = fm.FeatureCut4(*params)
            if feat is not None:
                break
        except Exception:
            continue
    if feat is None:
        raise RuntimeError("Cut-extrude failed.")
    return {"status": "ok", "message": f"Cut-extrude by {depth}m"}
```

Both flip directions are attempted because the cut direction relative to the sketch normal varies depending on which face the sketch was drawn on. The through-all variant uses end condition 1 (*swEndCondThroughAll*) in place of end condition 0 (*swEndCondBlind*) at parameter position 4.

C.2.9 Edge-Dependent Features

Fillets and chamfers require edges to be pre-selected using *SelectByRay*, with the append flag set to True for multi-edge selection.

```
def _fillet(radius):
    sw, doc = _get_active_doc()
    nv = _null()
    feat = doc.FeatureManager.FeatureFillet3(
        195, radius, 0, 0, 0, 0, 0, nv, nv, nv, nv, nv, nv, nv)
    if feat is None:
        raise RuntimeError("Fillet failed.")
    return {"status": "ok", "message": f"Fillet radius={radius}m applied"}

def _chamfer(distance):
    sw, doc = _get_active_doc()
    feat = doc.FeatureManager.InsertFeatureChamfer(
        4, 1, distance, 0, 0, 0, 0, 0)
    if feat is None:
        raise RuntimeError("Chamfer failed.")
    return {"status": "ok", "message": f"Chamfer distance={distance}m applied"}
```

The *FeatureFillet3* first parameter (195) is a bitfield constant combining *swFeatureFilletType_e* flags. Null dispatch variants are passed for the array parameters (radius array, set-back distances, point array) which are unused in the constant-radius case.

C.2.10 Material Assignment

```
def _set_material(material_name, database="solidworks materials.sldmat"):
    _, doc = _get_active_doc()
    doc.SetMaterialPropertyName2("", database, material_name)
    return {"status": "ok", "message": f"Material set to '{material_name}'"}
```

C.2.11 MCP Tool Registration

Each SolidWorks operation is exposed as an MCP tool using the *@mcp.tool()* decorator provided by the *FastMCP* framework. The decorator registers the function in the server's tool manifest, which Claude Desktop queries on connection. Parameter names and types are inferred from the Python function signature and exposed in the MCP tool schema as JSON.

```
mcp = FastMCP("SolidWorks Bridge")

@mcp.tool()
async def extrude_boss(depth: float, reverse: bool = False) -> str:
    """Extrude the last sketch as a Boss/Base feature. Depth in meters."""
    return json.dumps(await run_in_com(_extrude_boss, depth, reverse))

@mcp.tool()
async def sketch_circle_on_face(
        face_x: float, face_y: float, face_z: float,
```

```
        ray_dx: float, ray_dy: float, ray_dz: float,
        world_cx: float, world_cy: float, world_cz: float,
        radius: float) -> str:
    """Draw a circle on a model face using WORLD coordinates."""
    return json.dumps(await run_in_com(
        _sketch_circle_on_face, face_x, face_y, face_z,
        ray_dx, ray_dy, ray_dz, world_cx, world_cy, world_cz, radius))
```

Two representative tool registrations are shown above. The remaining 23 tools follow the same pattern: an async wrapper that calls *run_in_com()* with the corresponding blocking helper function.

C.2.12 Server Entry Point

```
if __name__ == "__main__":
    mcp.run(transport="stdio")
```

The server communicates with Claude Desktop over standard input/output using the MCP JSON-RPC protocol. Claude Desktop launches this script as a child process and manages its lifecycle automatically.